Real-time 2D Direction Steered Beamforming with a minimal 3 microphone array on a Resource-constrained FPGA

Anonymous EE2000 submission

Paper ID 4

Abstract

Directional hearing and assistive-listening devices benefit from knowing where a speech source is, but existing FPGA sound-localization systems typically use large microphone arrays, offload part of the pipeline to a hard processor or DSP, or report only coarse direction. This preliminary report surveys FPGA time-delay-of-arrival (TDOA) localization and beamforming designs, direction-of-arrival (DOA) algorithms, and hardware constraints in hearing-aid applications. We find that no reviewed design combines a minimal three-microphone array with fully embedded two-dimensional direction estimation on a small FPGA. We therefore propose an implementation and resource-characterization study: a processor-free RTL pipeline on a Xilinx Spartan-7 XC7S50 comprising I2S capture from three microphones, time-domain GCC-PHAT-style TDOA estimation for each microphone pair, azimuth estimation from the pairwise delays, and a DOA-steered delay-and-sum beamformer with real-time audio output and a bearing display. We will report LUT, DSP, and BRAM utilization against surveyed devices, together with angular accuracy. Stretch goals include a software baseline, a frequency-domain variant, adaptive nulling, and continuous tracking.

1. Introduction

We began our discussions with sound and acoustic signal processing applications, as there seemed to be a lot of scope in this area that was doable for us to cover. Eventually we narrowed it down to signal processing applications for hearing aids [29]. Of these, the most interesting was the ability to zoom into a particular direction, or in other words, find the direction of a speech source. Inspired by this, we decided to focus on minimal implementations of this concept on limited FPGAs, similar to the hardware constraints we are operating under. We attacked the related literature from different angles, separating the overall literature review into multiple pieces.

2. Foundational Sound Source Localisation (SSL) Algorithms & Their FPGA Implementations

The method of time-delay-based sound localization started from the Generalized Cross-Correlation (GCC) method of Knapp and Carter [25], whose PHAT weighting is still considered the standard way to estimate delays reliably in noisy, echoey rooms. An early FPGA system to use this idea was built by Nguyen et al. [35] in 2003: it was a two-microphone localizer that ran entirely on one chip and worked down to 10 dB SNR, with several such modules fitting on a single device. Over the next decade, three- and four-microphone FPGA designs mostly focused on making the hardware smaller and cheaper rather than more accurate. For example, a cross-correlation system with a separate azimuth stage [22], a soft-processor-based design using the MCCC algorithm [50], and a compact SOPC-based localizer [28], all reporting errors under about 3°. A related line of work looked at beamforming instead of localization, such as a fixed-null, two-microphone hearing-aid beamformer [42], which steers a null toward noise but does not track a moving source. The 2018 survey by Da Silva et al. [10] shows that FPGA microphone-array research after this point mostly scaled up to tens of microphones for delay-and-sum beamforming, rather than pushing small arrays further. In a recent FPGA-based Time-Delay-of-Arrival (TDOA) result we found, Jamaludin et al. [21], reports a mean angular error of $\pm 2.29°$ (97.46 % accuracy) on a DE1-SoC, though this was tested with a single tone rather than real speech, so it is not directly comparable to speech-based systems. Outside FPGA work, Kovalyov et al. [26] built the closest match to our goal, it is a three-microphone system that estimates 2D direction and tracks a moving source ,but on a smartphone rather than dedicated hardware, and Grondin et al. [16] showed that a simplified cross-correlation method can cut computation cost further on small microphone arrays, supporting our choice to avoid the frequency-domain approach (see Section 3).

Table 1. FPGA-based TDOA/SSL designs surveyed, in chronological order.

Work	Year	Algorithm	Mics	Device	Reported accuracy / result
Nguyen et al. [35]	2003	Modified-PHAT cross-correlation	2 (×5 parallel)	Virtex-II class	10 dB SNR floor; 77–108 mW/mic
Sound loc. system [22]	2008	Cross-corr. + energy + azimuth	3	Not reported	Real-time; 3°
MCCC DOA [50]	2013	MCCC (soft-core)	4	Cyclone II + Nios II	$< 3°$ deviation
Compact SOPC device [28]	2016	TDOA (SOPC)	Not reported	Cyclone III	$< 3°$ deviation
Fixed-null beamformer [42]	2017	Fixed-null beamforming (no DOA tracking)	2	Artix-7 (xc7a100t)	Resource utilization only (Vivado)
Da Silva et al. survey [10]	2018	Survey (multiple)	8–300 (surveyed)	Multiple	Survey; see Table 2–5 of source
Kovalyov et al. [26]	2023	mCPSP TDOA + NNS + tracking	3	Smartphone SoC (not FPGA)	Sub-lattice RMSE; qualitative tracking
Jamaludin et al. [21]	2024	TDOA	Not reported	DE1-SoC	$\pm 2.29°$ (97.46%)
This work	2026	Time-domain GCC-PHAT + 2D DOA steered beamforming	**3**	**Spartan-7 (XC7S50)**	*In progress*

3. FPGA Architecture and Resource-Efficient Implementation

Beyond the choice of localization algorithm, the surveyed FPGA-based systems diverge sharply in how much of the acoustic pipeline is delegated to a hard processor core versus implemented entirely in reconfigurable fabric. Sledevič and Laptik [44] demonstrate a representative SoC-class partitioning: their Xilinx Zynq-7020 design computes the six pairwise cross-correlations for a four-microphone GCC-TDOA array entirely in the programmable logic, while the embedded ARM Cortex-A9 core resolves the resulting delays into a 3-D source position, completing one full estimate in 28 ms. A similar split is used by Kowalczyk et al. [27], whose SoC FPGA performs demodulation and filtering on-chip before handing off SRP-PHAT localization and MVDR-based speech enhancement to the hard core, and by Azcarreta [3], whose 48-microphone Filter-and-Sum array uses its hard core only for network communication rather than for any signal-processing task. Takagi et al. [45] extend this pattern to a distributed setting, where each 16-element SoC-FPGA node runs a voice-activity detector and the MUSIC algorithm on-chip and reserves the hard core for inter-node communication. In contrast, an equally active line of work avoids a hard processor altogether. Petrica [37, 38] implements an 8- and later 16-element Delay-and-Sum beamformer entirely on bare fabric (Microsemi IGLOO2, then Xilinx Spartan-6), reporting 21.8 mW and 61.71 mW respectively with no host-side computation, and da Silva et al. [10] push this further to a 52-element array at 34 mW on a Flash-based SmartFusion2 device. Bare-fabric TDOA localization persisted at small scale as well: Biswas et al. [4] implement a coherence-based two-microphone TDOA locator entirely on a Xilinx Spartan-6, with no SoC or host involvement, as late as 2017. Thus, it was contemporaneous with, rather than preceding, the SoC-based designs above. The one three-microphone system in this body of work, the countersniper array of L'edeczi et al. [31], is itself bare-fabric (Xilinx XC2S100, using only a PicoBlaze soft-core for Bluetooth communication), but performs discrete shockwave/muzzle-blast event classification rather than continuous angle estimation. Taken together, this indicates that the SoC-versus-bare-fabric choice in the reviewed literature tracks array size and task complexity rather than chronology, and that no design – SoC or bare-fabric – combines a minimal three-microphone array and continuous, fully-embedded 2D DOA tracking on an extremely resource-constrained FPGA.

The choice between time-domain and frequency-domain beamforming fundamentally dictates the hardware footprint of FPGA sound locators, as detailed by da Silva et al. [11]. Frequency-domain processing forces the FPGA to execute multiple Multiply-Accumulate (MACC) operations to apply phase shift corrections, which mandates complex data representations using high bit widths or floating-point numbers [11]. To avoid the significant time and hardware resources demanded by the discrete Fourier transformation, time-domain Delay-and-Sum beamforming is widely preferred, as it eliminates these heavy MACC operations by simply using internal memory blocks to delay and synchronize incoming audio samples [10, 11]. This domain shift drastically alters the primary hardware bottlenecks. For instance, a high-performance 52-microphone architecture deployed on a Zynq 7020 FPGA is constrained primarily by logic, consuming 79.5% of the available Look-Up Tables (LUTs) and 41.8% of the DSP48 blocks. However, a power-efficient architecture on the same Zynq 7020 drastically reduces this logic footprint to just 7.6% for LUTs and 2.7% for DSP48s, shifting the primary constraint to internal memory by utilizing 18.9% of the BRAM18K resources [11]. When targeting a low-power Microsemi

SmartFusion2 M2S025 FPGA, this power-efficient design consumes exactly 5,547 4LUTs, 6 MACC (DSP) units, 216 RAM64x18 blocks, and 28 RAM1K18 blocks [11]. Overall, these established tradeoffs highlight how time-domain memory utilization replaces MACC logic complexity, providing the benchmark for us to choose time-domain processing on FPGAs that are primarily constrained in LUT count and other resource baselines.

4. Hardware constraints in assistive listening

Microphone-array design for hearing-aid and assistive-listening applications involves a trade-off between spatial selectivity, physical size, and computational complexity. Two-microphone directional systems have long been attractive because of their compact form factor; FPGA-based work has demonstrated adaptive beamforming with only two omnidirectional microphones while remaining suitable for real-time implementation [41, 42]. However, increasing the number of microphones can provide additional spatial degrees of freedom. Powers and Hamacher demonstrated a three-microphone behind-the-ear directional instrument, reporting an improvement in directivity over conventional two-microphone processing, while also identifying practical limitations arising from microphone mismatch, low-frequency attenuation, microphone noise, and the small spacing available in a hearing-aid form factor [39, 40]. More generally, microphone-array design for hearing aids is constrained by device dimensions, computational resources, and subject-dependent acoustic effects, which limit the practicality of simply scaling the array size [2]. At the other extreme, Inoue *et al.* [19] demonstrated a 48-microphone FPGA-based hat-type hearing-aid system, illustrating the directivity and steering flexibility achievable with a large aperture. The hat-based form factor, however, differs substantially from the compact physical constraints of ear-level hearing devices. Recent work has therefore increasingly focused on adaptive spatial processing rather than microphone count alone: multibeam hearing-aid systems can track multiple conversational partners in real time [13], while virtual-microphone methods seek additional spatial degrees of freedom from only a small number of physical microphones [12]. In this context, three microphones provide an intermediate design point: unlike conventional two-microphone beamformers, a non-collinear three-microphone geometry provides sufficient TDOA measurements for two-dimensional direction estimation, while avoiding the substantial physical and computational overhead of large arrays.

5. Comparison of DOA Algorithms

TDOA/DOA estimation is approached on accuracy, robustness, and computational grounds. Brutti *et al.* [5] compared GCC-PHAT against Adaptive Eigenvalue Decomposition (AED) and found AED more accurate under clean, non-interfering conditions. However, due to the iterative nature of its eigenvector estimation, AED was found to be considerably more computationally demanding than GCC-PHAT. This iterative loop makes it a poor fit for a fixed-resource FPGA pipeline, and GCC-PHAT matched or exceeded AED's accuracy on SNR below $15\,\mathrm{dB}$. Nguyen and Pham [36] compared classical cross-correlation (CC), the smoothed coherence transform (SCOT), GCC-PHAT, and SRP-PHAT for multi-source detection-localization. All three GCC-family methods were computationally cheaper than SRP-PHAT, though SRP-PHAT achieved better localisation precision at the cost of an exhaustive grid search. Such a method is not viable with a limited number of DSP slices. Kadhim and Khalifeh [23] benchmarked MUSIC, SRP-PHAT, WAVES, CSSM, FRIDA, and TOPS for hearing-aid type embedded systems, finding MUSIC and SRP-PHAT fastest ($0.030\,\mathrm{s}$ and $0.046\,\mathrm{s}$ respectively) while CSSM, WAVES, TOPS, and FRIDA required $0.47\,\mathrm{s}$ to $1.17\,\mathrm{s}$. However, methods involving subspace-decomposition and sparse-reconstruction must be discarded for real-time FPGA pipelines, since they reflect eigendecomposition or search costs that are extremely expensive on a 120-DSP-slice device. Finally, as recently as 2025, Choi *et al.* [8] proposed a filter-bank-weighted kernel density estimation as a noise-robustness extension of GCC-PHAT. This shows that GCC-PHAT is still actively used in research, and is not a superseded method.

One alternative we encountered deserves explicit acknowledgment: Ying *et al.* [51] propose a weighted minimum mean square error (WMMSE) estimator that avoids grid search entirely, reporting robustness close to SRP-PHAT at roughly five times the speed of GCC-PHAT and thirty times the speed of SRP-PHAT in software. WMMSE has not, to our knowledge, been validated in an FPGA or other embedded RTL implementation, so adopting GCC-PHAT (the method used by every hardware implementation reviewed in Section 2) carries lower implementation risk. However, we note WMMSE as a candidate for future work or a stretch goal due to its promising potential.

6. Minimal-Array Systems

Real-time 2D DOA tracking using a minimal three-microphone array is a well published direction, especially framed for hearing-aid style directional listening. Gu and Yang [17] demonstrated a three-microphone crosspower spectrum-phase system achieving 4.02° azimuth and 2.18° elevation accuracy. Independently from them, a long research program at the University of Texas at Dallas also progressed over 7 years. They began with an SVD-based non-uniform three-microphone smartphone system [14] through GCC and RSVD-based L-shaped-array imple-

Table 2. SoC-class vs. bare-fabric task partitioning in surveyed FPGA microphone-array designs.

Ref.	Year	Mics	On-FPGA fabric	On hard core / host	Power
[44]	2017	4	GCC cross-correlation (6 pairs)	3D position (28 ms)	N/S
[27]	2016	7	Demod. + filtering	SRP-PHAT + MVDR	N/S
[3]	2016	48	Filter-and-Sum beamform.	Network comms. only	N/S
[45]	2010	16	VAD + MUSIC + D&S	Inter-node comms.	N/S
[38]	2015	8	Full pipeline (bare, IGLOO2)	—	21.8 mW
[37]	2016	16	Full pipeline (bare, Spartan-6)	—	61.71 mW
[10]	2018	52	Full pipeline (bare, SmartFusion2)	—	34 mW
[4]	2017	2	Full TDOA pipeline (bare, Spartan-6)	—	N/S
[31]	2005	3	Shockwave/muzzle-blast detect (bare)	PicoBlaze soft-core (comms. only)	N/S

mentations. This was developed targeting hearing-aid users [46, 47]. Recently, their 2023 system [26] adds a learned time-difference-of-arrival (TDOA)-to-DOA mapping, multi-source clustering, and demonstrated tracking of a moving source. This covers most of the features that we plan to implement, plus some other additional ones. Finally, the most recent related work that we identified: Lopez-Olvera *et al.* [33] reported a four-microphone circular-array GCC-PHAT-and-delay-and-sum beamforming system explicitly designed for resource-constrained embedded platforms. They achieved sub-1° accuracy at 3.47 ms per-frame processing time. They published in August 2026, just a few weeks before this report. We took this as an indication that the field is still highly active.

We also analysed two broader surveys: Da Silva *et al.*'s [10] review of FPGA-based acoustic beamforming architectures has already been covered earlier. Other than that, there is Grinstein *et al.*'s [15] tutorial review of over 200 steered-response-power (SRP) papers. The latter noted that device-specific SRP implementations spanning CPUs, GPUs, and FPGAs achieve comparable performance. In conclusion, hardware realisations of these algorithms do exist, but they mostly target large mic arrays and SoC class platforms or general purpose DSP modules (like the XMOS chip used by Lopez-Olvera *et al.*), instead of working with bare, processor-less FPGA fabric.

Separately, Hu *et al.* [18] independently confirm that minimal three-pair microphone configurations yield relatively large estimation error relative to larger arrays, matching earlier quantitative findings by Klein and Vo [24], who showed three-microphone TDOA accuracy dropping to approximately 55% at $-10\,\mathrm{dB}$ SNR versus 99% for a ten-microphone array.

Taken together, we present this project as an implementation and resource characterisation study: realising this established three-microphone GCC-PHAT DOA steered beamforming pipeline entirely in RTL on a bare, SoC-less, processor-free FPGA, and reporting the resulting resource utilisation and accuracy trade-offs against the DSP and smartphone based implementations that otherwise dominate this space.

7. Motivation for a Bare FPGA Implementation

As the main contribution of our work, the value of a bare, SoC-less FPGA realization over the DSP and smartphone based implementations needs defending.

First, RTL implementations are the established precedent for migrating an assistive-listening algorithm toward a custom low-power ASIC. This was the path taken by Samtani *et al.* [42], whose FPGA hearing-aid beamformer was followed by an ASIC-optimized version of the same design the same year [41]. On the other hand, a software implementation on a commercial DSP module has no equivalent path.

Secondly, a fixed hardware pipeline offers cycle level deterministic latency and substantially lower power per operation than a general-purpose core running the same algorithm in software. These are both relevant parameters to a battery-powered wearable device. Finally, the parameter we wish to measure is that the actual size of this trade-off for the specific GCC-PHAT-and-delay-and-sum pipeline under study has not been measured anywhere in the reviewed literature.

8. Problem Statement

With the above analysis, we finally conclude the statement of our project.

8.1. Core Goals

- 3-microphone triangular array with I2S audio capture via a custom Register-Transfer Level (RTL) receiver
- Time-domain Phase Transform Generalized Cross-Correlation (GCC-PHAT)-style Time Difference of Arrival (TDOA) calculation for each microphone pair
- 2D Direction of Arrival (azimuth) estimation derived from the three pairwise TDOAs

- Live DOA-steered Delay-and-Sum Beamformer (DSB), outputting real-time audio through the onboard audio jack
- HDMI and/or LED-arc bearing indicator for visualisation
- Comprehensive resource utilization report (LUT, DSP, and BRAM consumption mapped against the physical limits of the Xilinx Spartan-7 XC7S50 FPGA), positioned against surveyed device benchmarks.

8.1.1 Novelty Claim

We characterise a 2D-DOA-tracked acoustic beamformer using a 3-microphone array on a severely resource-constrained device class currently absent from surveyed literature.

8.1.2 Next Steps

1. **Three-Microphone I2S Audio Capture**
   - **Study & Design:** Understand I2S protocol, microphone interfacing, clock generation, and RTL-based serial data reception.
   - **Implementation:** Design and implement the I2S receiver and microphone interface in Verilog/RTL, including synchronization and buffering.
   - **Testing & Validation:** Test captured samples against known audio/data patterns and understand reliable simultaneous acquisition from all three microphones.
2. **GCC-PHAT / TDOA Estimation**
   - **Study & Design:** Must look into cross-correlation, GCC-PHAT, phase transform, and methods for estimating TDOA between microphone pairs.
   - **Implementation:** Develop a hardware-efficient time-domain TDOA algorithm for the three microphone pairs.
   - **Testing & Validation:** Compare FPGA-derived TDOAs against software calculations and evaluate accuracy under different source positions and noise conditions.
3. **2D Direction-of-Arrival Estimation**
   - **Study & Design:** Build a system between the three pairwise TDOAs and the source azimuth for the triangular microphone geometry.
   - **Implementation:** Implement the azimuth estimation algorithm using the measured TDOAs.
   - **Testing & Validation:** Test the estimated bearing at different known source angles and calculate angular estimation error.
4. **Live DOA-Steered Delay-and-Sum Beamformer**
   - **Study & Design:** Study delay-and-sum beamforming, steering delays, interpolation/quantization requirements, and FPGA implementation constraints.
   - **Implementation:** Build the real-time beamformer and integrate it with the estimated DOA.
   - **Testing & Validation:** Evaluate the beamformer's ability to enhance the desired direction and suppress signals arriving from other directions.
5. **HDMI / LED-Arc Bearing Visualization**
   - **Study & Design:** Determine the required display/LED interface and design the mapping between estimated azimuth and visual bearing.
   - **Implementation:** Implement the HDMI /or LED-arc visualization system and integrate it with the real-time DOA output.
   - **Testing & Validation:** Verify that the displayed bearing follows the actual source position with minimal latency.
6. **FPGA Resource Utilization and Benchmarking**
   - **Study & Design:** Study the resource requirements of the proposed architecture.
   - **Implementation:** Synthesize and integrate the individual modules and obtain their resource utilization.
   - **Testing & Validation:** Generate a comprehensive utilization report and compare the obtained resource consumption with the surveyed device benchmarks, while identifying optimization opportunities.

8.2. Stretch Goals/Future Work

The following extensions will be implemented sequentially (arranged in order of novelty/feasibility) if the core goals are finished ahead of time. Otherwise, we will consider this for future work.

1. **Software Baseline Comparison:** Run the identical GCC-PHAT-and-delay-and-sum pipeline on a general-purpose low-power processor (such as a Raspberry Pi) and report latency/resource/power side-by-side with the FPGA RTL version, on the same 3-mic array.

2. **Frequency-Domain Variant:** FFT-based GCC-PHAT variant built in parallel with the time-domain pipeline to facilitate a direct empirical comparison of resource usage versus directional accuracy.

3. **Formal Accuracy Characterization:** Systematic experimental sweep quantifying angle estimation error against ground-truth angle, microphone array geometry/spacing, and Signal-to-Noise Ratio (SNR).

4. **Adaptive Nulling:** Basic adaptive null-steering oriented toward a secondary non-target direction, advancing beyond fixed-null baselines toward dynamic spatial filtering.

5. **Continuous Target Tracking:** Implementation of a tracking and smoothing filter (e.g., Kalman/Alpha-Beta) to follow moving speakers, replacing snapshot-only DOA estimates with continuous trajectory tracking.

6. **Directional Stereo Panning:** Implement stereo audio to acoustically show the direction of an audio signal to the user (as done in hearing aids that preserve binaural cues) instead of collapsing to mono as other literature does.

7. **WMMSE Implementation:** RTL implementation of the WMMSE as proposed by Ying *et al.* [51] and compare with our GCC-PHAT implementation.

9. Delay Estimation and Golden Model

This builds on Next Step-2 of Section 8.1.2.

Each microphone pair (A, B) must yield a delay, in samples, from which the source direction is computed (Sec. 11). This section defines the algorithm we implement, why we chose it, and the Python reference model the RTL will be verified against.

9.1. Algorithm

For one pair, $a[n] = s[n] + w_A[n]$ and $b[n] = \alpha\, s[n - D] + w_B[n]$, with source s, uncorrelated sensor noises w_A, w_B, attenuation α, and delay $D > 0$ when B hears the sound later (the RTL convention; Knapp and Carter's sign is opposite). The cross-correlation $r_{AB}[k] = E\{a[n]\, b[n + k]\} = \alpha R_{ss}[k-D]$ peaks at $k = D$, and the cross-spectrum has phase slope $-D$ [25]. Plain correlation has a limit: its peak has the width of the source autocorrelation (the inverse bandwidth), so peaks from reflections overlap and cannot be separated [25].

Generalised cross-correlation (GCC). GCC multiplies the cross-spectrum by a weighting $\psi(\omega)$ before returning to the lag domain,

$$r^{(\psi)}[k] = \tfrac{1}{2\pi}\int_{-\pi}^{\pi} \psi(\omega)\, \hat{G}_{AB}(\omega)\, e^{j\omega k} d\omega, \qquad (1)$$

$$\hat{D} = \arg\max_{k\in\mathcal{K}} r^{(\psi)}[k], \qquad (2)$$

with $\hat{G}_{AB}$ the cross-spectrum of an N-sample window and $\mathcal{K} = \{-13, \ldots, 13\}$ the lag range of our design. Every named processor/algorithm from here on is a choice of ψ: plain correlation ($\psi = 1$); SCOT ($1/\sqrt{G_{AA}G_{BB}}$); PHAT ($1/|G_{AB}|$), which for uncorrelated noise leaves the pure phase $e^{-j\omega D}$ and hence an ideal delta-shaped peak; and the maximum-likelihood (Hannan–Thomson) weighting, PHAT multiplied by $|\gamma|^2/(1 - |\gamma|^2)$ with γ the coherence, for which the accuracy grows with bandwidth, coherence and observation time [25]. The key caveat for usis that PHAT gives every frequency the same weight regardless of SNR, so it amplifies errors where the signal is weakest (and its phase is undefined where $G_{AB} = 0$). The ML weighting corrects this but needs the coherence, which must be estimated by averaging several segments

Time-domain realisation. The windowed correlation $\sum_n a[n]\, b[n+k]$ is a multiply–accumulate (MAC) over lags and needs no FFT. If both channels pass through the *same* FIR filter h, the delay phase is untouched and the correlator results in $\psi = |H|^2$. Two different filters leave a residual phase and bias $\hat{D}$. "Time-domain GCC-PHAT" is not exactly an established algorithm, so we use the term only for our own purposes to refer to a lag correlator with a fixed shared prewhitening filter. The nearest precedent to what we have chosen to do is the multichannel cross-correlation (MCCC) method [6], used for array direction finding [24] and implemented on a soft-core FPGA [50]. With $\tilde{a}, \tilde{b}$ the (optionally whitened) samples, $\tilde{b} = 0$ outside the window, the score and estimate are

$$s[k] = \sum_{n=0}^{N-1} \tilde{a}[n]\, \tilde{b}[n + k], \qquad \hat{k} = \min\arg\max_{k\in\mathcal{K}} s[k], \quad (3)$$

where min means ties go to the lower lag. *Version 1* (V1 of the golden model) uses the raw samples. *Version 2* (V2) uses the saturated first differencing filter $\tilde{x}[n] = \mathrm{sat}_{16}(x[n]-x[n-1])$, i.e. $\psi_d = 4\sin^2(\omega/2)$, which whitens a source whose spectrum falls as $1/\omega^2$; it costs one subtraction per sample and is a separate, swappable RTL module if we later dcide on a separate whitening. It also boosts high frequencies, where noise tends to dominate.

Multichannel extension. Chen et al. generalise the correlation coefficient to $L+1$ microphones via spatial prediction and pick the delay that minimises the determinant of

the normalised correlation matrix [6, 7]. For two microphones this is plain correlation. More microphones sharply cut anomalous estimates , but the gain at *three* microphones is modest: at -5 dB their two- and three-microphone estimates were both one sample off and four microphones were exact. With no spectral weighting the method should inherit plain correlation's reverberation sensitivity; combining it with prewhitening is untried in the sources we saw, and neither did we manage to test it ourselves. Since our triangular array would need the source angle instead of a single lag parameter, we kept three pairwise estimates and use the sum rule $\hat{\tau}_{13} \approx \hat{\tau}_{12} + \hat{\tau}_{23}$ as a consistency check.

One-bit quantisation. For jointly Gaussian zero-mean signals with correlation ρ, the correlation betwewen their single-bit signs follows an arcsine relationship (Van Vleck–Middleton [49]; used for one-bit DOA by Liu and Vaidyanathan [32]). Consequences of this include: (i) arcsin is monotone, so the one-bit correlation peaks as the same location as the true one and no inversion is needed; (ii) the score is $s[k] = N_k - 2m_k$ (N_k is overlapping samples, m_k is sign mismatches), computed easily with XNOR logic and a bit population count, with no multiplier; (iii) Quantising down to one bit increases estimation variance by at least $2.47\times$ (3.9 dB), comparable to the 4–6 dB MSE loss in [32] The law is exact only for Gaussian signals and DC offsets bias the sign, so DC must be removed before quantisation and the one-bit path needs to be separately validated empirically on speech-like data.

9.2. Golden model

9.2.1 RTL specification

For one pair the correlator takes $N = 1024$ signed 16-bit samples per channel and evaluates $s[k]$ for the 27 lags $k \in \{-13, \ldots, 13\}$; $k > 0$ means B is later. Products are 32 bits; the worst-case sum $N(2^{15})^2 = 2^{40}$ needs a 42-bit signed accumulator. One shared multiplier loops over lags then samples: $27 \times 1024 = 27{,}648$ MAC cycles per pair and window. The output, once per window, is the best lag and the scores at $\hat{k} - 1, \hat{k}, \hat{k} + 1$ (a neighbour outside $\mathcal{K}$ is output as 0). Pairs are $(1, 2), (1, 3), (2, 3)$; V2 resets its filter state to 0 at every window. The RTL passes if it equals the golden model exactly on every window and pair.

9.2.2 Implementation and verification

`golden.py` implements V1, V2 and an FFT GCC-PHAT reference (zero-padded to 2048 samples), each at three precisions: *full* (float64), *16-bit* (exact integer arithmetic as in the RTL) and *1-bit* (no fixed-point FFT is modelled though). We checked it with an independent exporter recomputing every record with separate code (`np.correlate`) and stopping on any mismatch.

SNR (dB)	Method	Full	16-bit	1-bit
10	V1 / V2 / FFT-PHAT	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0
0	V1	4.2	4.2	23.7
	V2	32.3	32.3	55.8
	FFT-PHAT	10.5	10.5	34.8
-5	V1	39.8	39.8	63.7
	V2	82.0	83.0	88.2
	FFT-PHAT	53.3	53.3	72.5

Table 3. Delay estimates (%) that miss the true lag (white noise, no reverberation).

The test families cover negative, zero and edge (± 13) lags, full-scale constants and random samples, all zeros, noisy coloured sources at about 10 and 0 dB, and a three-window file that checks the resetting rule for V2 .

On the 432 simulated cases of Sec. 12 (V1, V2 and their one-bit variants), golden and independent implementations agreed on every record and no overflow flag was raised. The largest score needed 39 bits (V1), 35 bits (V2) and 11 bits (one-bit), against the 42 provisioned; 42 remain necessary (and is feasible on the Boolean Board, though we could not compile the full resource utilisation report in time) for full-scale input, but speech peaking near 30000 needs 3 bits fewer. The best lag hit the edge of the ± 13 range in none of the V1 cases but in 10 of 432 V2 cases (25 for one-bit V2), confirming that the first difference creates weird peaks at high lag values.

Preliminary results: precision and weighting in noise. We compared the methods on a synthetic signal (AR(1) source with pole 0.9, rms 4000) with integer delays 0, 3, 6 samples at the three microphones , independent white noise per microphone, no reverberation. Table 3 gives the percentage of estimates (300 trials $\times$ 2 pairs, $N = 1024$, lags ± 13) whose lag is not exactly the true lag (standard error $\lesssim 2$ points).

In the white-noise, non-reverberant setting weighting actually hurts us: V1 beats FFT-PHAT, which beats V2. This we considered as consistent with Knapp and Carter's remark on PHAT amplifying errors where the signal is weak. The same results for echoic rooms is tested in Sec. 12.

10. Sub-sample Peak Interpolation

This is to clarify part of Next Step-2 in Section 8.1.2.

Now, integer lags are not enough most of the time. A true delay is rarely a whole number of samples, so $\hat{k}$ is off by up to half a sample, an RMS error of $1/\sqrt{12} \approx 0.29$ sample [9]. For a pair of spacing d the delay is $(d/c)\cos\theta$, so near broadside one sample is $\Delta\theta \approx c/(df_s)$: 5.1° (8 cm) and 8.2° (5 cm) at an *assumed* $f_s = 48$ kHz, giving integer-only RMS errors of about 1.5° and 2.4° per pair, grow-

ing as $1/\sin\theta$ towards endfire. At 16 kHz in our simulations one sample is 15.4° (8 cm), 19.2° (6.4 cm) or 24.6° (5 cm), and the largest lag is only 3.7 samples. These are the size of the errors reported for three-microphone systems (a few degrees), so sub-sample resolution is needed. The real sampling rate was still to be fixed from the microphone datasheet at the time of testing, so to cover all bases, Secs. 10–11 use 48 kHz, Sec. 12 uses 16 kHz.

10.1. Literature

Cespedes et al. split methods into *curve fitting* (fit a parametric peak to a few lags around $\hat{k}$) and *reconstructive* (rebuild the band-limited correlation between lags) [9]; Maskell and Woods estimate a phase directly, for periodic signals only [34]; Laakso et al. supply the fractional-delay (FD) filter theory and advise trying simple methods first (Lagrange of length 2 or 4) [30]. The standard, cheapest method is a three-point parabola [9, 20]. With $u = s[\hat{k}] - s[\hat{k}-1] \geq 0$ and $v = s[\hat{k}] - s[\hat{k}+1] \geq 0$,

$$\hat{\delta} = \frac{s[\hat{k}+1] - s[\hat{k}-1]}{2\big(2s[\hat{k}] - s[\hat{k}+1] - s[\hat{k}-1]\big)} = \frac{u-v}{2(u+v)}, \quad \hat{D} = \hat{k}+\hat{\delta}. \tag{4}$$

It uses exactly the three scores the golden model already outputs. Because the centre sample is the largest, $|\hat{\delta}| \leq \frac{1}{2}$ always, and the fraction can come from a shift-subtract divider (one bit per clock, no DSP slice); $\hat{\delta}$ must be forced to 0 when $\hat{k}$ is at the edge of $\mathcal{K}$.

10.2. Accuracy limits

Both fits are biased because the true peak is neither a parabola nor a cosine [9, 20]. For a cosine correlation the parabolic estimate is $\hat{\delta} = \frac{T_s}{2}\cot(\frac{\omega T_s}{2})\tan(\omega\delta)$. The bias vanishes at $\delta = 0, \pm 0.5$ peaks near $|\delta| \approx 0.3$, depends on the peak shape (so no fixed correction removes it), grows with bandwidth, inflates the Cramér–Rao bound [25] by at most about 26% in their cases, and rises abruptly when the three samples no longer lie in the main lobe (rule $f_s \geq 6f_0$). For a moving source it shows up as an angle ripple with a period of one sample of delay. Interpolation is therefore a refinement of an already correct integer lag. Reconstructive (windowed-sinc) interpolation is unbiased, independent of peak shape, and equivalent to zero-padding the cross-spectrum but cheaper; in the ultrasound case of [9] more than 12 taps and about 350 interpolated points reach the Cramér–Rao bound.It still needs more lags than the three scores the golden model outputs (interpolating $s[k]$ at a fractional lag is the same operation as fractionally delaying a sequence). For the beamformer's steering delays, Lagrange is maximally flat and accurate at low frequency, the Farrow structure changes the delay every sample with a fixed filter bank (second-order polynomials usually suffice), and Thiran IIR allpass filters need divisions, can be unstable and give transients when the delay changes [30]; that design is kept for the beamformer. The quadrature estimator [34] correlates one channel with in-phase and quadrature references (a quarter-period shift) for $2N+2$ multiplications and one arctangent; its simulated bias stays near zero while a parabola keeps about $2 \times 10^{-3}\,T$ even at 30 dB, but it presumes a periodic single-frequency signal and does not apply to speech.

max |mean err|	parab	cos	sinc4	lin	P^2	tri
lp25 V1	.011	.008	.025	–	–	–
lp25 V2	.012	.006	.025	–	–	–
AR(1) V1	.072	.069	.075	–	–	–
lp25 V1, 1-bit	.099	.093	–	.019	.016	.015
AR(1) V1, 1-bit	.132	.127	–	.070	.068	.050

Table 4. Noiseless worst-case bias (samples). "lin": parabola after $\sin(\pi c/2)$; "P^2": parabola on $-P^2$; "tri": two-line fit.

	SNR	int	parab	cos	sinc4	P^2 / tri
lp25 V1	30	.276	.008	.007	.020	–
	10	.284	.036	.036	.045	–
	0	.360	.187	.186	.252	–
lp25 V2	30	.276	.009	.008	.020	–
	10	.357	.136	.134	.218	–
	0†	.716	.607	.598	.662	–
AR(1) V1	30	.293	.052	.050	.056	–
	10	.283	.060	.059	.063	–
	0	.365	.203	.202	.229	–
lp25 1-bit	30	.278	.070	–	–	.035 / .035
	10	.297	.090	–	–	.087 / .096
	0	.487	.327	–	–	.325 / .313

Table 5. RMS delay error (samples; 300 trials; wrong-integer-lag trials excluded). †13.7% anomalous trials excluded.

10.3. Testing

`subsample.py` first reproduces the literature's bias results on ideal shapes: maximum near $|\delta| = 0.3$, rapid growth below $f_s/f_0 \approx 6$, and a worst-case parabolic bias of $0.0065\,T_s$ at $f_s/f_0 = 10$ on a cosine correlation, the same order as Cespedes' $\approx 0.01\,T_s$ (for a Gaussian-envelope case we get $0.009\,T_s$ against roughly $0.03\,T_s$ read from their figure, unreconciled). It then measures the interpolators on our correlator score ($k \in [-13, 13]$, $N = 1024$, true delay $3+\mu$, $\mu \sim U(-0.5, 0.5)$, independent white noise per channel, 300 trials per cell) for two sources: band-limited flat noise cut at 25% of Nyquist ("lp25", speech-like oversampling) and the AR(1) source of Sec. 9, which has energy up to Nyquist. "sinc4" is windowed-sinc reconstruction with 8 taps and 4 bisection steps. Results are in Tables 4 and 5.

On oversampled speech-like data the parabola is accurate to about 0.01 sample (0.06° at 8 cm broadside); on the

non-oversampled AR(1) source it is 0.07 sample, consistent with the $f_s \geq 6f_0$ rule. The sinc result (0.025) is limited by its 1/16 grid and is no better on AR(1). For one-bit data the plain parabola is badly biased (0.10). In noise, integer-only RMS is 0.28–0.29 sample, as predicted. At 30 dB the parabola and cosine fit reach 0.008 sample on lp25 (0.04° at 8 cm, 0.07° at 5 cm, per pair at broadside); at 10 dB 0.036 (0.18°, 0.29°); at 0 dB 0.19 (0.96°, 1.5°), much better than the integer 0.36. Parabola and cosine fit are practically identical and sinc is never better, so the extra lags and arithmetic are not justified. V2 is clearly worse in white noise (13.7% anomalous lags at 0 dB). The one-bit corrections help only at high SNR (0.070→0.035 at 30 dB).

11. Transforming Pairwise Delays to Direction

This builds on Next Step-3 of Section 8.1.2.

Two papers treat this problem: Gu and Yang [17] match delays against a pre-computed table; Kovalyov et al. [26] compare a closed-form inversion with a learned nearest-neighbour table (NNS).

For calculations, we took an equilateral array of side $a = 8$ cm (mic 1 at the origin, mic 2 at $(a, 0)$, mic 3 at $(a/2, \sqrt{3}a/2)$).

11.1. Geometry and far-field model

The array is planar; the source direction is $\mathbf{u} = (\cos\phi\cos\theta, \cos\phi\sin\theta, \sin\phi)$ (azimuth θ from the mic 1→2 axis, elevation ϕ). A non-collinear array removes the front–back ambiguity of a single pair. Let $\tau_{ij} = \|\mathbf{s} - \mathbf{m}_j\| - \|\mathbf{s} - \mathbf{m}_i\| = (c/f_s)D_{ij}$ be the path difference (positive when j hears later; both papers use the opposite sign). In the far field $\tau_{ij} \approx -\mathbf{u} \cdot (\mathbf{m}_j - \mathbf{m}_i)$, so with $b = a$, $c_x = a/2$, $c_y = \sqrt{3}a/2$:

$$\tau_{12} = -b\,u_x, \quad \tau_{13} = -(c_x u_x + c_y u_y), \quad \tau_{23} = \tau_{13} - \tau_{12}. \tag{5}$$

The last relation holds exactly at any distance. Hence only two delays are independent (the third is a check), and since all microphones have $z = 0$ the delays do not depend on u_z: a planar array cannot tell above from below, and the range cannot be found. This is why both papers estimate direction only, and why our core goal is just finding azimuth.

11.2. Deriving Closed-form inversion

Solving (5) gives $\hat{u}_x = -\hat{\tau}_{12}/b$, $\hat{u}_y = (c_x\hat{\tau}_{12} - b\hat{\tau}_{13})/(b\,c_y)$ and $\hat{\theta} = \operatorname{atan2}(\hat{u}_y, \hat{u}_x)$. For the equilateral layout the common factor cancels:

$$\hat{\theta} = \operatorname{atan2}\big(\hat{\tau}_{12} - 2\hat{\tau}_{13},\ -\sqrt{3}\,\hat{\tau}_{12}\big). \tag{6}$$

For a source at known distance r the step is exact: squaring $\|\mathbf{s} - \mathbf{m}_j\| = r + \tau_{1j}$ gives

$$s_x = \frac{b^2 - 2r\tau_{12} - \tau_{12}^2}{2b}, \tag{7}$$

$$s_y = \frac{c_x^2 + c_y^2 - \tau_{13}^2 - 2r\tau_{13} - 2c_x s_x}{2c_y}, \tag{8}$$

$$s_z = \sqrt{r^2 - s_x^2 - s_y^2}, \tag{9}$$

which is the closed form of Kovalyov et al. [26] (they fix a large constant r and take $s_z \geq 0$, since a planar array cannot tell above from below), and $\mathbf{s}/r \to \hat{\mathbf{u}}$ as $r \to \infty$. Properties of the far-field form: (i) since $\mathbf{u}_{xy} \propto (\cos\theta, \sin\theta)$ for every elevation, the azimuth is correct whatever ϕ is; (ii) combining two non-parallel baselines avoids the two mirror solutions and the $1/\sin\theta$ endfire blow-up of inverting one pair with arccos; (iii) a common scale error (in c, f_s or array size) cancels in atan2; (iv) for in-plane sources $\tau_{12}^2 + \tau_{13}^2 + \tau_{23}^2 = 3a^2/2$ and elevated ones give less, so an estimate outside this ellipse is anomalous.

11.3. Gu and Yang [17]

For every direction on a 5° azimuth–elevation grid they pre-compute the three delays from exact spherical propagation (1297 entries) and output the grid direction closest to the measured delay vector; the per-frame angles of one voice command are combined by an energy-weighted mean (short-time energy beat RMS amplitude as the weight). Their delays come from an FFT cross-power-spectrum-phase estimator with parabolic refinement. The table's source distance is not stated. Set-up: right-triangle array with 20 cm legs (our reading of "20 cm between each pair"), $f_s = 16$ kHz, loudspeaker replaying speech at 1.0 and 1.5 m, azimuth −90° to 90° in 30° steps, elevation −30°, 0°, 30°. Mean error 4.02° azimuth, 2.18° elevation: 2.1°–3.4° for $|\theta| \leq 60°$ but up to 5.7° at ±90° at 1.0 m, and 11.0° and 7.1° at 1.5 m (blamed on the non-linear delay–angle relation and a fan).

11.4. Kovalyov et al. [26]

They note that the closed form needs exact geometry and ignores microphone phase responses, and instead learn the mapping in the field: a loudspeaker at 0.6 m on a turntable gives delay triples that are interpolated onto 10^4 equal-area (Fibonacci) lattice points; the estimate is the nearest stored triple, found with a k-d tree in expected logarithmic time (against 1297 distance evaluations for a full table scan, and a few multiplications plus one atan2 for the closed form). In their noise study NNS is best at low and moderate noise, the closed form with hand-measured geometry is clearly worst (its flat curve at low noise indicates a floor set by geometry mismatch, not delay noise), and calibrating the geometry recovers part of the gap. They use $f_s = 48$ kHz and 1024-sample frames with 50% overlap (as our golden model),

and reject frames whose correlation peak is too low or not dominant, or whose NNS residual exceeds $T_q = 5 \times 10^{-5}$ ($\sqrt{T_q} = 7$ mm, about one sample at 48 kHz); our closure residual is the analogous delay-domain check. Their clustering stage serves multiple sources and is not needed for one. Neither paper reports a degree error for a closed-form mapping, so we evaluate ours in the delay-to-angle study.

11.5. Accuracy Limits

(i) *Delay noise versus aperture.* Gu and Yang's $4.02°$ on 20 cm corresponds to about $41\,\mu$s (0.65 sample at 16 kHz); the same angle on 8 cm needs $16\,\mu$s, i.e. 0.56–0.96 sample at 48 kHz depending on azimuth. Our interpolation results (0.04–0.19 sample RMS at 10–0 dB, white noise, no echo) lie below this, so delay noise alone should not prevent a few-degree error; reverberation and the effects below are not covered.

(ii) *Geometry (first order).* An x-placement error δ of mic 3 biases azimuth by $\approx (\delta/c_y)\cos^2\theta$ (up to $1.7°$ for 2 mm); length errors of the two baselines by at most $0.75|\delta_{13} - \delta_{12}|/c_y$ ($0.6°$ for 1 mm on one baseline); a common scale error cancels. Any single-microphone error is of order δ/d rad, consistent with the floor of the uncalibrated closed form in [26].

(iii) *Finite range.* Applying the far-field inversion to a source at distance r gives, for an in-plane source and first order in a/r,

$$\hat{\theta} - \theta \approx \tfrac{a}{4\sqrt{3}\,r}\big[4\sin(\theta - 30°) - \cos 3\theta\big], \qquad (10)$$

up to $2.4°$ at $r = 1$ m and $1.2°$ at 2 m for $a = 8$ cm, as large as the best reported accuracies; the exact-range form (9) with a nominal r or a correction from (10) is needed. Gu and Yang's exact-delay table is immune apart from range mismatch.

12. Replication: Weighting Under Noise and Echo

This is to verify our math in Section 9.

We replicate the literature claim that PHAT weighting keeps a delay estimator accurate in noise, on our own 3-microphone geometry.

12.1. Literature claim

Knapp and Carter present PHAT as the *sharpest-peak* weighting (a delta for uncorrelated noise), call it ad hoc, and note that it amplifies errors where signal power is low; noise robustness requires the coherence term of the ML weighting [25]. Brutti et al. found GCC-PHAT better than adaptive eigenvalue decomposition (AED) below about 15 dB SNR on real recordings (7 microphones, 32 cm spacing, 48 kHz, $T_{60} \approx 0.65$ s, noise added to give 30–2 dB; AED has the lower RMSE on clean data), but they did not compare against plain correlation [5]. Nguyen and Pham, comparing plain correlation (CC), SCOT and GCC-PHAT on the AV16.3 8-microphone array, found CC best (average standard deviation $4.47°$ against $4.65°$ for both others) and cheapest, with the SCOT advantage only at low noise [36]. We test: **H1** PHAT stays at least as accurate as CC as SNR falls; **H2** PHAT beats CC in a reverberant room; **H3** SCOT and PHAT behave similarly.

12.2. Set-up

Simulated data (image-source method [1], pyroomacoustics [43]): speech from a point source 1.5 m from a 3-microphone triangle (sides 8, 5, 6.4 cm) in a $6 \times 5 \times 3$ m room, $f_s = 16$ kHz. 36 angles (0–350°) $\times$ 3 echo levels ($T_{60} = 0, 0.25, 0.5$ s) $\times$ 4 SNRs (30, 20, 10, 0 dB) give 432 windows of $N = 1024$ (the loudest stretch of speech) and 1296 pairwise lags; noise is independent white Gaussian per microphone (checked: $|\rho| < 0.06$ at 0 dB). The truth is the direct-path lag rounded to a sample; all true lags lie in $[-4, 4]$. Estimators use the form of (2) with the same sign convention and a ± 13 lag search: **CC** ($\psi = 1$, float FFT); **SCOT** (auto-spectra smoothed by Welch averaging, 256-sample Hann segments, untuned); **PHAT**; **V1**, **V2** (16-bit) and **V1 1-bit** of Sec. 9; and **PHAT 0.3–4 kHz** (our diagnostic: PHAT with $\psi = 0$ outside that band). V1 is the integer version of CC and returned the same lag in all 1296 estimates. An estimate counts as correct if $|\hat{k} - D| < 1$ (one of the two integers around the true delay; 17.6% of true lags lie within 0.1 sample of a rounding boundary). Each cell has 108 estimates (95% interval up to ± 9.4 points), optimistic since pairs share microphones and SNR levels reuse the same speech, so we use paired comparisons to claim differences.

First case: No echo. CC is correct in 100% of cases at 10 dB and above (94.7% overall; 78.7% at 0 dB). PHAT and SCOT drop to 37.0% and 56.5% at 0 dB, below CC in every noisy cell, so **H1 is not reproduced**. Both even *lose* accuracy from 20 to 30 dB, which noise alone cannot explain. The cause is visible in the spectra: speech power falls by about 40 dB from below 300 Hz to 6–8 kHz, and the inter-microphone coherence in 30 dB cases is about 0.99 at 0.3–1 kHz but only 0.06–0.30 above 4 kHz. PHAT weights these noise-dominated bins as heavily as the clean ones, which is the weakness Knapp and Carter noted. Restricting PHAT to 0.3–4 kHz confirms it: it is correct in 100% of cases at 20 and 30 dB (92.6% at 10 dB), though still behind CC at 0 dB (50.0% against 78.7%). One unexplained feature: all SCOT and PHAT errors at 30 dB , and 47–52% of their errors overall (24% for CC), fall at lag exactly 0; in the cases examined the 6–8 kHz band alone peaks at lag 0, which suggests a property of the simulated data rather than of the estimators.

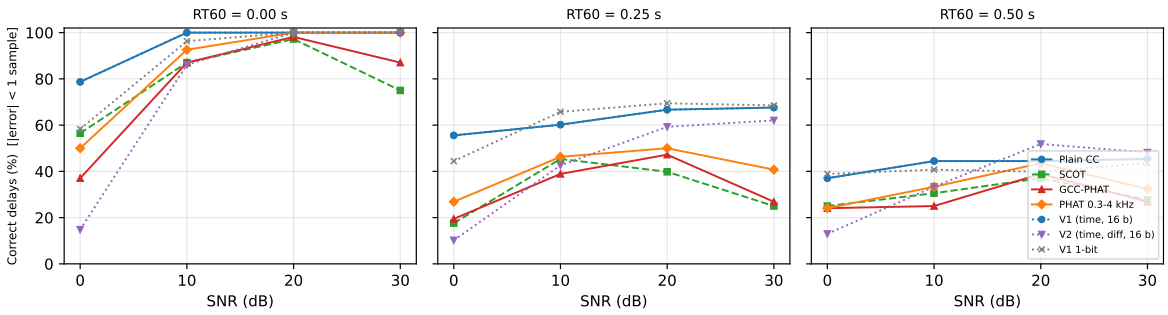


Figure 1. Correct delays ($|\hat{k} - D| < 1$) versus SNR for each echo level (108 estimates per point; the V1 curve lies under plain CC; ± 13 lag search).

Table 6. Correct delays (%), ± 13 lag search, best per row in bold; 108 estimates per cell. CC is identical to V1.

T_{60} (s)	SNR (dB)	CC / V1	SCOT	PHAT	PHAT 0.3–4 kHz	V2	V1 1-bit
0.00	0	**78.7**	56.5	37.0	50.0	14.8	58.3
	10	**100.0**	87.0	87.0	92.6	86.1	96.3
	20	**100.0**	97.2	98.1	**100.0**	**100.0**	**100.0**
	30	**100.0**	75.0	87.0	**100.0**	**100.0**	**100.0**
0.25	0	**55.6**	17.6	19.4	26.9	10.2	44.4
	10	60.2	45.4	38.9	46.3	42.6	**65.7**
	20	66.7	39.8	47.2	50.0	59.3	**69.4**
	30	67.6	25.0	26.9	40.7	62.0	**68.5**
0.50	0	37.0	25.0	24.1	24.1	13.0	**38.9**
	10	**44.4**	30.6	25.0	33.3	33.3	40.7
	20	44.4	37.0	38.9	43.5	**51.9**	39.8
	30	45.4	27.8	26.9	32.4	**48.1**	43.5

Table 7. Marginals over SNR (432 estimates per row): correct delays (%); overall exact match with the rounded lag; fraction outside the physical range $|\hat{k}| > 4$.

T_{60} (s)	CC/V1	SCOT	PHAT	PHAT-BL	V2	1-bit
0.00	**94.7**	78.9	77.3	85.6	75.2	88.7
0.25	**62.5**	31.9	33.1	41.0	43.5	62.0
0.50	**42.8**	30.1	28.7	33.3	36.6	40.7
all	**66.7**	47.0	46.4	53.3	51.8	63.8
exact match, all	**40.5**	28.2	29.2	31.1	33.5	36.6
$\lvert\hat{k}\rvert > 4$ (%)	0.2	5.8	6.5	6.3	16.5	0.8

We will check our simulator regarding this.

Second case: With echo. Echo dominates (Fig. 1). At $T_{60} = 0.25$ s CC is correct in 62.5% of cases overall (67.6% at 30 dB) and at 0.5 s in 42.8% (45.4%); curves are flat above 10 dB, so the error is echo-limited, not noise-limited. PHAT (33.1%, 28.7%) and SCOT (31.9%, 30.1%) are far lower (band-limited PHAT recovers some: 41.0%, 33.3%), so **H2 is not reproduced**: on the 1296 paired estimates PHAT is right where CC is wrong in 60 cases and the reverse in 323 (sign test $p \approx 10^{-44}$; 45 against 180 for reverberant cases at $\geq$ 10 dB; SCOT 50 against 305). CC changes by only 7 points from 10 to 30 dB at $T_{60} = 0.25$ s, against 21 points from 0 to 10 dB without echo. This agrees with Nguyen and Pham [36]. **H3 holds overall** (47.0% against 46.4%), although individual cells differ by up to 20 points. A plausible but untested reason that PHAT does not help: with 5–8 cm spacing at 16 kHz every reflection falls within about 4 samples of the direct path, leaving no room for a sharper peak to separate them. At our chosen 48 kHz (Section14) results should be better.

Third case: Time-domain versions and lag range. The 16-bit integer correlator loses nothing against floating point (V1 equals CC). V2 is worse than V1 almost everywhere (51.8% against 66.7% overall; 12.7% against 57.1% at 0 dB) and 16.5% of its estimates fall outside the physical range, against 0.2% for V1; it is ahead only at $T_{60} = 0.5$ s and $\geq$ 20 dB, within the cell intervals. One-bit costs little where it matters: 63.8% against 66.7% overall, 9.9 points at 0 dB, at most 1 point at 10 dB and above, supporting a

multiplier-free path at moderate SNR [49]. Restricting the search to ± 4 changes CC by only $+0.2$ points but raises SCOT, PHAT and V2 by 2.0, 2.0 and 5.1 points, because their noisy peaks lie outside the physical range. For the RTL this is free: $K = 4$ needs $9 \times 1024 = 9216$ MAC cycles per pair and window instead of 27,648.

13. Design Decisions So Far

1. **Delay estimator:** plain correlation (V1) in 16-bit integer arithmetic (never more than 7.5 points below the best estimator in any of the 12 echo/SNR cells); the "GCC-PHAT-style" wording of the problem statement is changed to be "correlation-based". V2 can be kept as a swappable module. We won't pursue coherence-weighted ML variant because it needs segment-averaged spectra.
2. **Lag window:** sized it to the sampling rate, ± 13 at 48 kHz (max delay 11.2 samples for 8 cm)
3. **Sub-sample:** add the three-point parabola (4) on the scores the correlator already outputs, with a 4–6 bit shift-subtract divider (resolution 2^{-5} sample, about 0.16° at 8 cm broadside, 48 kHz) and $\hat{\delta} = 0$ at the window edge.Interpolation only helps when the integer lag is already right (SNR above about 5 dB, low echo): at $T_{60} \geq 0.25$ s the integer lag is wrong in 38–57% of cases.
4. **Delay to angle:** closed-form far-field inversion (6): a shift, a subtraction, one constant multiplication and an atan2 (CORDIC or small table), with no search and no stored map. Use the third pair as a validity check (closure residual, ellipse test), smooth in the delay domain rather than on angles , fix the geometry mechanically (bias about δ/d rad), and keep a 1° azimuth table as fallback (360×3 delays, serial search in about 10^3 cycles, small next to $3 \times 27{,}648$ correlator cycles; it doubles as a reference for [17]).

14. Three-Microphone I2S Capture Front-End

14.1. Scope and approach

This part reports progress on Next Step 1 of Section 8.1.2 (three-microphone I2S audio capture). At the time of writing and understanding the theory the INMP441 microphone modules were not yet confirmed to be available. We therefore built the capture chain to be *microphone independent*: a parameterized receiver is verified against an RTL model of a microphone (an "emulator") that plays stored samples in the I2S format, so the full clock–emulator–receiver chain can be tested in simulation and be uploaded on the Boolean Board without a physical microphone. All results in this section come from RTL simulation; hardware validation with the real microphones is still pending.

14.2. Architecture

Figure 2 shows the capture chain. It consists of three RTL modules, all running on the single 100 MHz board clock.

Clock divider (`clk_div`). The divider derives the two signals every general I2S microphone needs. With a half-period of 16 system clocks, BCLK $= 100\,\text{MHz}/32 = 3.125$ MHz. With 32 BCLK cycles per channel slot, WS $= \text{BCLK}/64 \approx 48.83$ kHz, which is the sample rate. One divider serves all three microphones, so the three channels share the same frame timing. BCLK and WS are ordinary registers rather than derived clocks; the divider additionally emits one-cycle pulses (`bclk_rise`, `bclk_fall`, `ws_edge`) so the rest of the design stays in one clock domain. As in standard I2S, WS changes on the falling edge of BCLK.

Receiver (`i2s_rx`). Each microphone has its own data line (three input pins in total), and one shared controller drives three 24-bit shift registers. Each data line first passes through a two-flip-flop synchronizer. When WS changes, the receiver checks whether the new slot is the one the microphones use (left, WS$= 0$, with the INMP441 L/R pin tied to ground). The first BCLK rising edge after the WS change is discarded, because in I2S the most significant bit appears one BCLK *after* the WS transition. The next 24 rising edges shift the data in, MSB first; the receiver then keeps the top 16 of the 24 bits, raises a one-cycle `valid` pulse, and ignores the padding bits and the other slot until the next WS change. The three 16-bit outputs are therefore updated in the same clock cycle, which keeps the channels time-aligned, a requirement for the pairwise TDOA estimation that follows.

Microphone emulator (`mic_emu`). The emulator behaves like one microphone: it sees only BCLK and WS, detects their edges itself, and drives its data line in the correct slot with the correct one-BCLK delay. It plays 24-bit samples from a 256-entry table initialised from a file. A short Python script generates the three tables from the same two-tone signal, with channel 1 delayed by 2 samples and channel 2 by 5 samples relative to channel 0, imitating the inter-microphone delays that the TDOA stage will later measure.

14.3. Verification and preliminary results

A self-checking testbench connects the divider, three emulators and the receiver and compares every received sample with the corresponding stored value (top 16 of the 24 bits). It ran for 300 frames, which is more than one pass through the 256-entry table, so the wrap-around is also exercised. Table 8 summarizes the outcome.

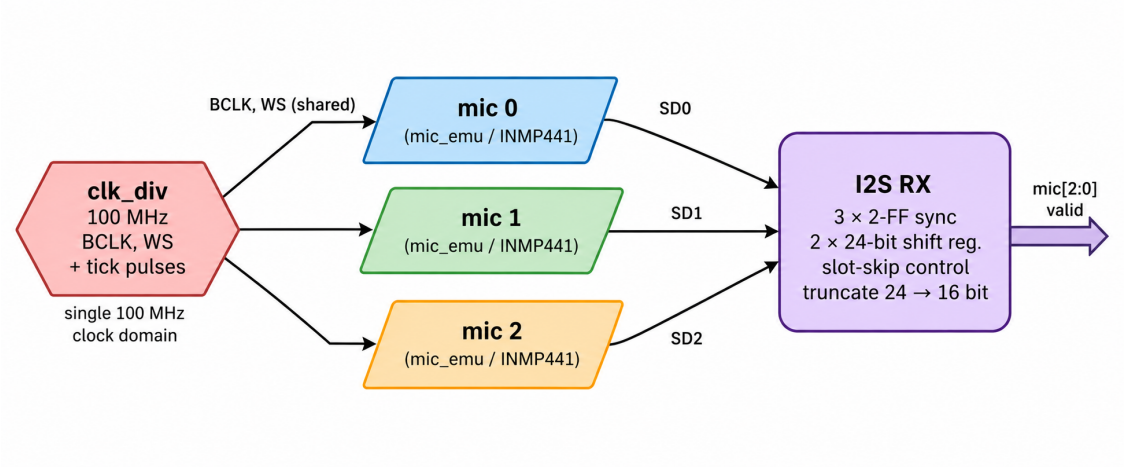


Figure 2. Capture chain. One clock divider supplies BCLK and WS to all three microphones (or emulators); each microphone returns its own data line to the receiver, which outputs three time-aligned 16-bit sample streams. The divider's tick pulses also feed the receiver (not drawn).

Table 8. RTL simulation results (Icarus Verilog), divider → 3 emulators → receiver.

Quantity	Result
BCLK / WS frequency	3.125 MHz / 48.83 kHz
Frame period	20.48 μs
Samples compared (per mic)	299
Mismatches (3 channels)	0
`valid` pulses	one per frame
Wrong bit injected	299 mismatches
One-BCLK skip removed	test fails

The fault-injection rows show that the testbench is sensitive to the two most likely errors (a corrupted data bit, and the classic off-by-one-BCLK I2S misalignment), so the passing result is meaningful. Figure 3 shows one received sample from the same simulation: the 24 bits sampled on the BCLK rising edges read `0x18F660` and the receiver outputs the top 16 bits, `0x18F6`, together with a single `valid` pulse.

14.4. Status and next steps

Completed: clock divider, receiver, microphone emulator, self-checking testbenches and the sample-generation script (all submitted with this report). The things remaining before the final report are : (i) confirm the timing details against the INMP441 datasheet and the real module; (ii) add the microphone pin constraints for the Boolean Board expansion connector (the clock constraint is already in place); (iii) run the chain on the board and report the synthesis utilization of these modules (Next Step 6); and (iv) connect the three sample streams to the time-domain GCC-PHAT stage.

15. Beamforming Theory and Experimental Evaluation

16. Delay-and-Sum Beamforming

Van Veen and Buckley [48] describe beamforming as a way to filter sound based on its direction. The signals from the microphones are delayed or phase-adjusted and then added together. For sound arriving from the selected direction, these adjustments make the signals line up, so they add more strongly. Sound from other directions may not line up as well and can be reduced.

A simple time-domain expression for delay-and-sum beamforming is

$$y(t) = \frac{1}{M} \sum_{m=1}^{M} x_m(t - \tau_m), \tag{11}$$

where M is the number of microphones, $x_m(t)$ is the signal from microphone m, and τ_m is the delay applied to that signal. The delays depend on the microphone positions and the direction we want to listen to.

For a narrowband signal, the beamformer can also be written as

$$y[n] = \mathbf{w}^H \mathbf{x}[n], \tag{12}$$

where $\mathbf{x}[n]$ is the vector of microphone samples, $\mathbf{w}$ is the complex weight vector, and $(\cdot)^H$ means conjugate transpose.

For an ideal far-field plane wave, the normalized magnitude response of an equal-weight delay-and-sum beam-

EE2000
#4

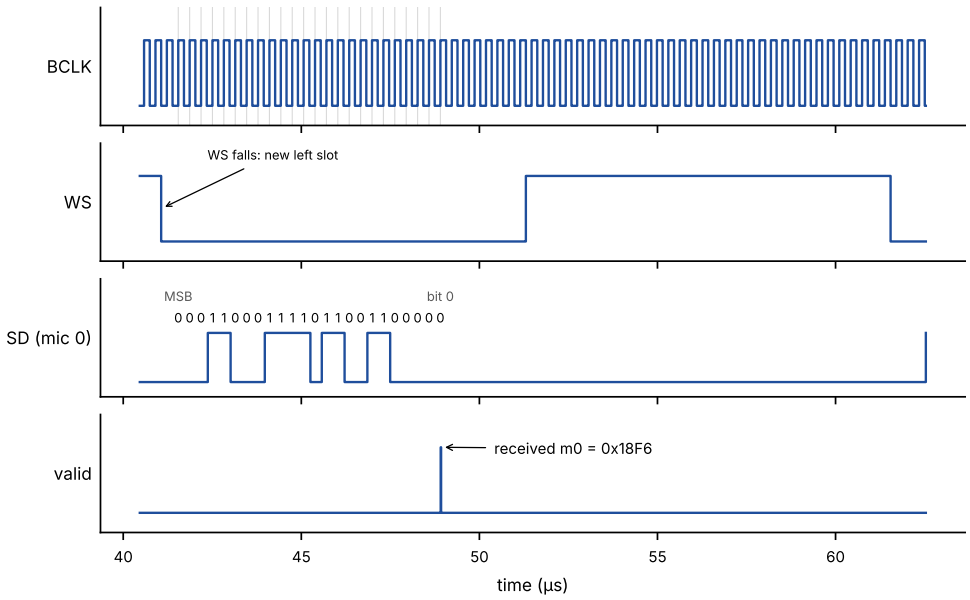


Figure 3. Simulation waveform of one sample on microphone 0. The digits above SD are the values the receiver samples on each BCLK rising edge (the first rising edge after WS falls is skipped). The receiver outputs `0x18F6`.

former is

$$B(f,\theta) = \frac{1}{M}\left|\sum_{m=1}^{M} \exp\left[j\frac{2\pi f}{c}\mathbf{r}_m^T\left(\mathbf{u}(\theta)-\mathbf{u}(\theta_0)\right)\right]\right|, \tag{13}$$

where f is frequency, c is the speed of sound, $\mathbf{r}_m$ is the position of microphone m relative to the array centre, θ is the incoming direction, and θ_0 is the direction towards which the beamformer is steered. The direction vector is $\mathbf{u}(\theta) = [\cos\theta, \sin\theta]^T$.

The response in decibels is

$$B_{\mathrm{dB}}(f,\theta) = 20\log_{10}\left(B(f,\theta)\right). \tag{14}$$

The response is normalized so that the steering direction has a magnitude of one, or 0 dB.

The wavelength is

$$\lambda = \frac{c}{f}. \tag{15}$$

The size of the array compared with the wavelength affects how well it can distinguish directions. If the array is small compared with the wavelength, the phase differences between microphones are small and the beam pattern is usually broad. A larger aperture can improve direction selectivity, but it can also create unwanted lobes or direction ambiguities [48]

Powers and Hamacher [39] discuss compact directional microphone systems for hearing aids. Their design uses a linear delay-and-subtract arrangement, which is different from our triangular equal-weight delay-and-sum array. They report low-frequency attenuation of about 6 dB per octave for a first-order subtractive system and about 12 dB per octave for their second-order three-microphone system. They also show that microphone mismatch can reduce directivity. These results explain some general challenges of compact microphone arrays, but the exact attenuation figures do not directly apply to our array.

17. DOA Accuracy and Related Work

Time-difference-of-arrival (TDOA) methods estimate the time difference between the signals received by microphones. These delays can then be used to estimate the direction of the source. Noise can make the delay harder to estimate. Reverberation can also cause errors because reflected sound may be mistaken for the direct sound.

Klein and Võ [24] study the multichannel cross-correlation coefficient (MCCC) method. It uses information across several microphones together. For each possible alignment, it builds a normalized spatial correlation matrix and uses its determinant to choose the delay estimate. This is not the same as GCC-PHAT. GCC-PHAT normalizes the magnitude of a pairwise cross-spectrum, while MCCC uses a multichannel determinant-based criterion.

In a reverberation-free simulation at -10 dB SNR, Klein and Võ report about 55% correct delay estimates for three microphones and over 99% for ten microphones. Their method measures whether the estimated integer-sample delay is exactly correct. Our software experiment uses a different estimator and counts a direction estimate as correct

if it is within 10° of the true direction. The percentages therefore cannot be compared directly.

Brutti, Omologo, and Svaizer [5] compare several sound-source localization methods using real recordings. Their work is useful for understanding that localization results depend on the algorithm and recording conditions. It also provides context for correlation-based methods such as GCC-PHAT and other spatial-response approaches.

18. Pyroomacoustics Simulation and Audio Output

We use Pyroomacoustics to simulate sound propagation in a room. Its image-source method models reflections as sound arriving from virtual copies of the source behind the walls. The microphones therefore receive both direct sound and delayed reflections. This lets us test delay estimation for different source directions, noise levels, and room conditions.

In the supplied validation results, the estimated delays matched the reference exactly in 24.1% of microphone-pair cases and were within one sample in 92.6% of cases when reflections were included. In the direct-path-only test, all 108 cases matched exactly. These results show that reflections can cause delay estimation errors. They are separate from the DOA accuracy results in Sec. 20.

The simulator used a $6 \times 5 \times 3$ m room, a sampling rate of 16 kHz, and a source about 2 m from the array. The initial test used synthetic smoothed noise. The microphone geometry also differs between experiments. The simulator used $(0, 0)$, $(0.08, 0)$, and $(0, 0.05)$ m, giving side lengths of 8, 5, and 9.43 cm. Tasks 7–8 used $(0, 0)$, $(0.08, 0)$, and $(0.03, 0.04)$ m, giving side lengths of 8, 5, and about 6.4 cm. Therefore, the results from these setups should not be treated as coming from the same triangle.

The hardware must also send the beamformer output to the Boolean Board audio jack. The FPGA output format and the board's audio interface and output circuitry must be checked to ensure that the digital signal can be played as audio.

19. Python Delay-and-Sum Beamformer

The Python beamformer applies delays using phase shifts in the frequency domain rather than directly delaying the signals in time. For each candidate direction, it calculates the phase shifts based on the microphone positions and combines the signals. It then selects the direction with the highest output power.

This is a frequency-domain steered-response search, and this version merely provides a software baseline before the FPGA implementation.

Table 9. DOA accuracy at -10 and $+10$ dB SNR. An estimate is correct if it is within $\pm 10^\circ$ of the true direction.

RT_{60}	Array	-10 dB	$+10$ dB
0 s	3-mic triangle	12%	98%
0 s	10-mic circle, $r = 5$ cm	31%	100%
0 s	10-mic circle, $r = 10$ cm	65%	100%
0.4 s	3-mic triangle	8.5%	33%
0.4 s	10-mic circle, $r = 5$ cm	16.5%	55%
0.4 s	10-mic circle, $r = 10$ cm	25.5%	57%

20. Accuracy Versus SNR

We used Pyroomacoustics to compare three array configurations:

- Three microphones in a triangle at $(0, 0)$, $(0.08, 0)$, and $(0.03, 0.04)$ m.
- Ten microphones on a circle of radius 5 cm.
- Ten microphones on a circle of radius 10 cm.

The source was placed 1.5 m from the array centre. A speech recording sampled at 16 kHz was divided into 1024-sample windows. Forty high-energy windows were selected. The original experiment used 200 randomized trials for each array, SNR, and room condition. SNR ranged from -10 to $+10$ dB in steps of 2 dB.

We tested direct-path propagation with reflections disabled and a reverberant room targeting $RT_{60} = 0.4$ s. The estimator searched angles from 0° to 359° and selected the direction with the highest summed output power.

The circular angular error was calculated as

$$e_\theta = \left| \left[(\hat{\theta} - \theta + 180^\circ) \bmod 360^\circ \right] - 180^\circ \right|, \tag{16}$$

where θ is the true direction and $\hat{\theta}$ is the estimated direction. An estimate was counted as correct if $e_\theta \le 10^\circ$. Accuracy was

$$A = 100 \frac{N_{\text{correct}}}{N_{\text{trials}}} \%. \tag{17}$$

This is the percentage of estimates within a 10° tolerance. It is not the mean angular error or the exact integer-delay success rate used by Klein and Võ.

20.1. Results

In the direct-path condition, the ten-microphone arrays perform better than the three-microphone triangle at low SNR. As shown in Table 9 and Fig. 4, at -10 dB, the triangle has 12% accuracy, compared with 31% for the 5 cm-radius circle and 65% for the 10 cm-radius circle. At $+10$ dB, all three arrays reach at least 98% accuracy.

EE2000
#4

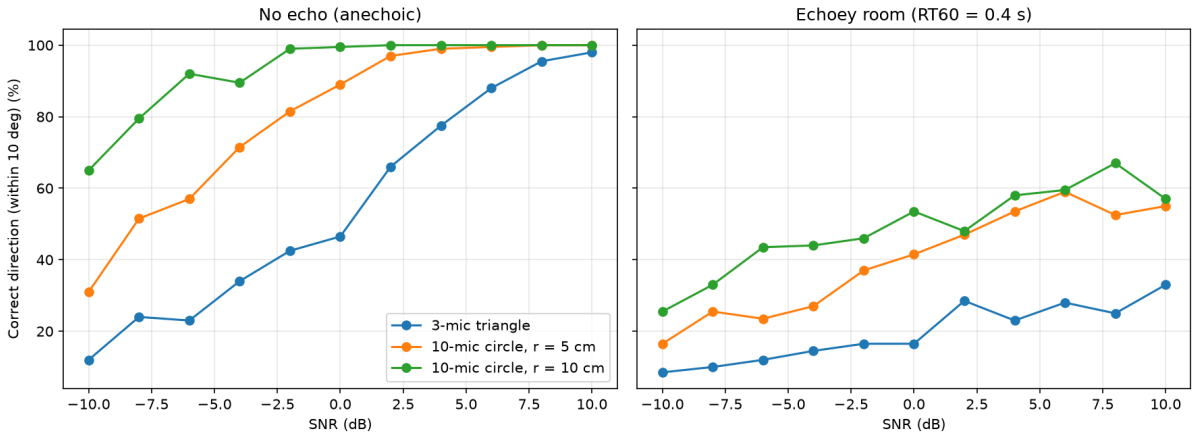


Figure 4. DOA accuracy versus SNR for the three-microphone triangle and the two ten-microphone circular arrays. Left: direct path only. Right: simulated room targeting $RT_{60} = 0.4$ s.

Reverberation lowers accuracy for all three arrays, as shown in Table 9 and Fig. 4. At -10 dB, the accuracies are 8.5%, 16.5%, and 25.5%, respectively. At $+10$ dB, they are 33%, 55%, and 57%. This shows that reflections remain a major problem even when additive noise is lower.

The trend is consistent with the general observation in Klein and Võ [24] that more microphones can improve delay-estimation robustness. However, our reverberant results are much lower than their reported delay-estimation results. The setups differ in several ways: their ten-microphone linear array spans about 99 cm, while our circular arrays have diameters of 10 cm and 20 cm; their estimator is MCCC, while ours is a frequency-domain steering search; their metric is exact integer delay, while ours is angle error within $\pm 10°$; and their room conditions are not the same as our RT_{60} simulation.

The small aperture and the estimator's response to reflections may contribute to our lower results, but the current experiment does not tell us which factor is responsible for how much of the difference. Also, the ten-microphone arrays change both microphone count and array aperture, so the experiment does not isolate microphone count alone. Thus, we are treating these results as findings for our simulation, and not exactly a replication of their experiment.

21. Frequency-Dependent Beam Patterns

We also calculated ideal beam patterns for the three-microphone triangle. The beamformer was steered to 0°, and we measured the response to an interferer at 90° at 500 Hz, 1 kHz, 2 kHz, and 4 kHz. These calculations assume a far-field plane wave, equal microphone weights, perfectly matched microphones, and no room reflections. The resulting patterns are shown in Fig. 5, and the response values are summarized in Table 10.

Table 10. Ideal normalized response at a 90° interferer.

Frequency	Wavelength	Response at 90°
500 Hz	68.6 cm	−0.60 dB
1 kHz	34.3 cm	−2.53 dB
2 kHz	17.15 cm	−9.83 dB
4 kHz	8.575 cm	−0.40 dB

At 500 Hz, the wavelength is much larger than the array. The longest microphone spacing is only about 0.12λ, so the phase differences are small and the beam pattern is broad, as shown in Fig. 5. The response at 90° is only −0.60 dB (Table 10), which means that the array provides very little rejection in that direction.

At 1 kHz, rejection at 90° is still limited. At 2 kHz, the response drops to about −9.83 dB, giving stronger rejection. At 4 kHz, the response at 90° rises again to about −0.40 dB, although the pattern has a deeper null at another angle. These changes are visible in Fig. 5 and show that the array does not necessarily become more directional at every angle as frequency increases.

At 4 kHz, the wavelength is about 8.58 cm, close to the longest microphone spacing of 8 cm. Phase wrapping and an unwanted lobe are plausible reasons for the response at 90°. However, the array is a non-uniform triangle, so we should not claim that spatial aliasing is definitely the only cause.

One very important clarification here is that these are theoretical narrowband plots, while speech is broadband, so its performance depends on the array response across many frequencies.

Powers and Hamacher's low-frequency attenuation figures refer to a subtractive array, not our equal-weight delay-and-sum array. On the other hand, our results show that the

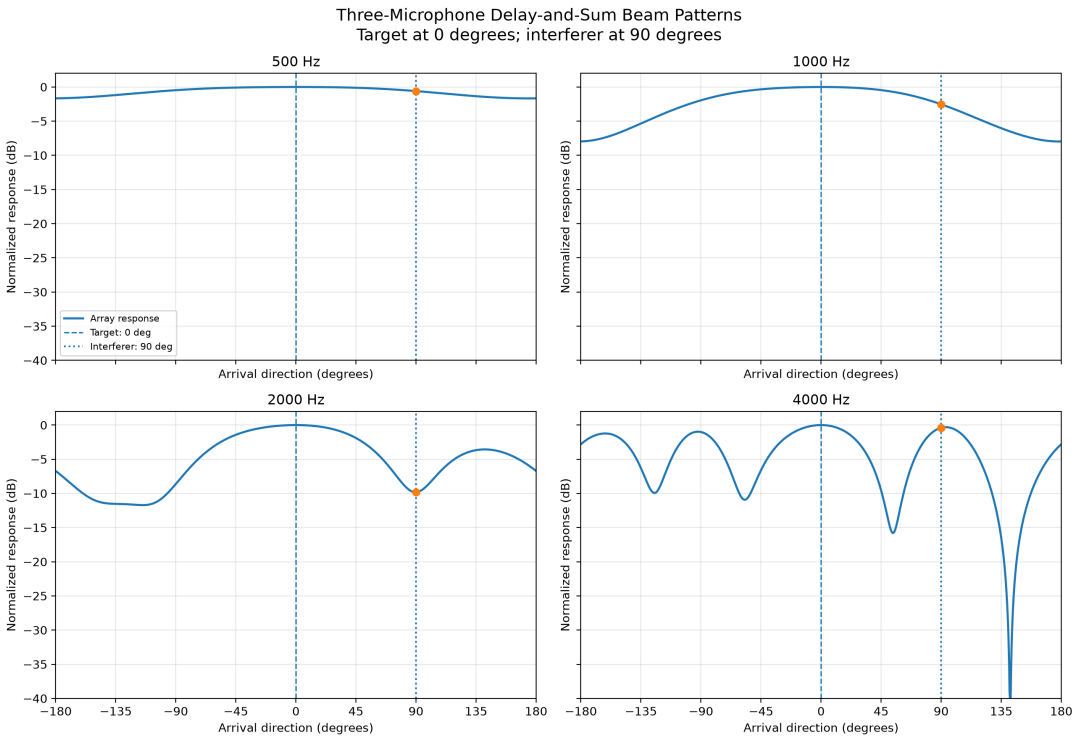


Figure 5. Ideal far-field delay-and-sum beam patterns for the three-microphone triangular array. The beam is steered to 0°.

small aperture relative to wavelength makes low-frequency direction identification difficult.

22. Hardware Implementation of Time-Domain Correlation and Delay Estimation

This work is based on Next Step-2 of Section 8.1.2, building on and implementing the work done in Section 9.

22.1. Objective and System Overview

The delay finder serves as an intermediate digital signal processing stage positioned directly between the I2S audio receiver and the delay-to-angle mapping block. Its core function is to estimate the integer sample delay (lag) at which audio signals from a pair of microphones exhibit maximum cross-correlation. To process the entire sensor array, three parallel instances of the module are deployed to compute pairwise delays τ_{12}, τ_{23}, and τ_{31}. The system receives 16-bit two's-complement audio streams sampled at $f_s = 48$ kHz ($T_s \approx 20.8\,\mu$s). Assuming an acoustic speed of sound $c \approx 343$ m/s, one sample period corresponds to approximately 7.15 mm of differential path length. The module outputs the detected optimal lag, correlation agreement scores at the peak lag and its immediate neighbors, an edge-condition flag, and a processing completion pulse.

22.2. Array Geometry and Search Range

22.2.1 Spatial delays

The microphone array comprises three sensors configured in an equilateral triangle with side length $d = 8$ cm and circumradius $R = d/\sqrt{3} \approx 4.62$ cm. In Cartesian coordinates, the microphone positions are defined as $p_1 = R(1, 0)$, $p_2 = R(-1/2, \sqrt{3}/2)$, and $p_3 = R(-1/2, -\sqrt{3}/2)$. For a far-field acoustic source arriving from azimuth direction $u = (\cos\theta, \sin\theta)$, the inter-element spatial separation d produces a maximum theoretical propagation delay of

$$|\tau|_{\max} = \frac{d}{c} = \frac{0.08}{343} \approx 233\,\mu\text{s}, \tag{18}$$

which corresponds to 11.2 samples at 48 kHz. A positive delay sign convention $\tau_{ij} > 0$ indicates that microphone j receives the wavefront later than microphone i, satisfying $(p_i - p_j) \cdot u = c\tau_{ij}$. Physical consistency across the array requires that the delay sum closes to zero, $\tau_{12} + \tau_{23} + \tau_{31} \approx 0$.

22.2.2 Search window and boundary logic

To account for mechanical placement tolerances, clock jitter, and thermal variations in sound speed, the correlation search range is expanded to MAXLAG $= 13$, yielding

$2 \times 13 + 1 = 27$ discrete evaluation lags. Lags detected at the boundaries ($|k| = 13$) fall outside the theoretical physical limits and trigger an edge flag to signal downstream processing stages that sub-sample peak interpolation should be skipped. Lags beyond $[-13, +13]$ are never evaluated. When the optimal lag occurs at best_lag $= -13$, the left neighbor score score_m1 is forced to zero; similarly, when best_lag $= +13$, the right neighbor score score_p1 is set to zero.

22.3. Time-Domain 1-Bit Sign Correlation

Mathematical rationale. Standard cross-correlation computes $r[k] = \sum_n y[n]x[n-k]$ to identify peak delay $\hat{D} = \arg\max_k r[k]$. While generalized cross-correlation with phase transform (GCC-PHAT) provides sharp delta-like peaks by frequency-domain normalization, its direct implementation requires complex FFTs, complex multiplications, divisions, and inverse FFTs. These operations demand substantial logic resources and dedicated DSP slices on field-programmable gate arrays (FPGAs) such as the Xilinx Spartan-7 XC7S50.

To eliminate hardware multipliers, the initial design adopts a time-domain 1-bit cross-correlation architecture. By quantizing each 16-bit sample to its sign bit (± 1), the Van Vleck–Middleton arcsine law guarantees that for Gaussian signals,

$$E[\text{sign}(x)\,\text{sign}(y)] = \frac{2}{\pi}\arcsin(\rho), \tag{19}$$

where ρ is the true underlying signal correlation. Because $\arcsin(\rho)$ is strictly monotonic, the peak location remains unchanged after 1-bit quantization. In digital hardware, multiplying two sign bits reduces to a single XNOR gate, and window accumulation simplifies to bit-counting logic.

22.4. Hardware architecture

22.4.1 History registers and indexing

Rather than recomputing full window sequences on every sample, the module maintains compact shift registers and running accumulators. Shift register x_sr stores $2M + 1 = 27$ history bits and y_sr stores $M + 1 = 14$ history bits, where $M = \text{MAXLAG} = 13$. New sign bits enter index 0 on every valid clock pulse. Bit $y_sr[M]$ serves as a stationary reference sample against which $x_sr[0 \ldots 2M]$ bits are compared in parallel via XNOR gates. Register index $i \in [0, 26]$ maps directly to lag $k = i - M$, establishing $i = 0$ as lag -13, $i = 13$ as lag 0, and $i = 26$ as lag $+13$. History registers are maintained continuously across window boundaries without resetting, eliminating warm-up overhead after the initial window.

22.4.2 Finite state machine

Control logic is structured around a three-state finite state machine (FSM):

1. **Accumulate (ACC):** On each incoming valid sample pulse, 27 running accumulators and a sample counter are updated simultaneously. When the sample counter reaches $N = 1024$, state transitions to SCAN.

2. **Scan (SCAN):** Over 27 clock cycles, a sequential comparator identifies the maximum agreement score best_val and its corresponding index best_idx. A strict greater-than comparison ensures that score ties resolve deterministically to the lowest index (most negative lag).

3. **Output (OUT):** The FSM calculates best_lag $=$ best_idx $- M$, latches peak score score_0, neighbor scores score_m1 and score_p1, and the boundary edge flag, asserts a single-cycle DONE pulse, clears accumulator counters, and returns to ACC.

The state sequence is formally defined in Algorithm 1. The complete output sequence finishes in 29 clock cycles, which is negligible compared to the approximately 2000 clock cycles available between 48 kHz audio arrivals.

22.5. Verification Strategy

Module verification incorporates a self-checking testbench architecture evaluating pseudo-random streams, zero-fill vectors, and synthetic delayed inputs across zero delay, positive delay, negative delay, and extreme boundary lags (± 13). Hardware outputs are cross-verified against a Python reference golden model (Section 9.2) using test memory files (`.mem`). Global system validity across the three parallel hardware instances is validated by confirming that pairwise estimated delays satisfy the spatial closure condition $\tau_{12} + \tau_{23} + \tau_{31} \approx 0$ across all valid tracking conditions.

22.6. Experimental Results and Design Revision

Evaluation on basic vs. speech-like test cases. Initial functional verification using standard, noise-free synthetic signals yielded 100% estimation accuracy across all baseline test vectors. However, when evaluated on realistic room-simulated acoustic datasets featuring speech signals across varying reverberation times (RT60) and additive signal-to-noise ratios (SNR), overall estimation accuracy severely degraded to 17.82% (77 passed out of 432 total test cases). The detailed breakdown of test performance across acoustic conditions is summarized in Table 11.

Algorithm 1 `delay_finder` Rising Clock Edge Behavior

Parameters: W (bit width), N (window size), M (MAXLAG); $MAXLAG_2 \leftarrow 2M + 1$

done $\leftarrow 0$
if *rst* **then**
  state $\leftarrow$ ACC; count $\leftarrow 0$; scan_i $\leftarrow 0$
  $x_sr \leftarrow 0$; $y_sr \leftarrow 0$; clear all outputs
  acc$[i] \leftarrow 0$ for $i = 0 \ldots MAXLAG_2 - 1$
else
  if *valid* **then**
    $x_sr \leftarrow \{x_sr[MAXLAG_2{-}2 : 0],\ x[W{-}1]\}$
    $y_sr \leftarrow \{y_sr[M - 1 : 0],\ y[W - 1]\}$
  end if
  if state $=$ ACC **then**
    if *valid* **then**
      for $i \leftarrow 0$ **to** $MAXLAG_2 - 1$ **do**
        if $x_sr[i] == y_sr[M]$ **then**
          acc$[i] \leftarrow$ acc$[i] + 1$
        end if
      end for
      if count $== N - 1$ **then**
        count $\leftarrow 0$; scan_i $\leftarrow 0$; state $\leftarrow$ SCAN
      else
        count $\leftarrow$ count $+ 1$
      end if
    end if
  else if state $=$ SCAN **then**
    if scan_i $== 0$ **or** acc[scan_i] $>$ best_val **then**
      best_val $\leftarrow$ acc[scan_i]; best_idx $\leftarrow$ scan_i
    end if
    if scan_i $== MAXLAG_2 - 1$ **then**
      state $\leftarrow$ OUT
    else
      scan_i $\leftarrow$ scan_i $+ 1$
    end if
  else if state $=$ OUT **then**
    best_lag $\leftarrow$ best_idx $- M$
    score_0 $\leftarrow$ acc[best_idx]
    score_m1 $\leftarrow$ (best_idx $==$ 0) ? 0 : acc[best_idx $- 1$]
    score_p1 $\leftarrow$ (best_idx $== MAXLAG_2 - 1$) ? 0 : acc[best_idx $+ 1$]
    edge_flag $\leftarrow$ (best_idx $== 0$) **or** (best_idx $== MAXLAG_2 - 1$)
    done $\leftarrow 1$; acc$[i] \leftarrow 0$; state $\leftarrow$ ACC
  end if
end if

22.6.1 Architectural revision

Analysis of the failure cases indicates that 1-bit quantization via sign-bit (MSB) extraction is highly susceptible to phase distortion caused by multipath reverberation and non-Gaussian speech dynamics. While anechoic cases (RT60 $=$ 0.00 s) achieved somewhat higher accuracy of 51.39%, introducing moderate reverberation (RT60 $\geq$ 0.25 s) corrupted the sign bit alignment, driving accuracy down near zero regardless of SNR.

Table 11. Performance Summary of 1-Bit Sign Correlation Model

Category / Parameter	Value / Rate
SUMMARY STATS	
Total Cases Tested	432
Total Passed	77
Total Failed	355
Overall Accuracy	17.82%
ACCURACY BY RT60 (Echo Level)	
RT60 $=$ 0.00 s	74 / 144 (51.39%)
RT60 $=$ 0.25 s	3 / 144 (2.08%)
RT60 $=$ 0.50 s	0 / 144 (0.00%)
ACCURACY BY SNR (Noise Level)	
SNR $=$ 30 dB	28 / 108 (25.93%)
SNR $=$ 20 dB	30 / 108 (27.78%)
SNR $=$ 10 dB	16 / 108 (14.81%)
SNR $=$ 00 dB	3 / 108 (2.78%)

To overcome these limitations, we have revised the architecture to replace 1-bit MSB sign correlation with full 16-bit signed multiplication across channels. This design change trades off computational resources—utilizing FPGA DSP slices to compute full two's-complement product accumulations—in exchange for preserving signal magnitude dynamics and restoring acceptable delay estimation accuracy in real-world acoustic environments. **Testing with the new model is still in progress, but it has not yet given significantly better results. Table 12 shows the preliminary results we have obtained so far, and we are actively working on further improving this model.**

22.7. Delay-to-Angle Mapping and System-Level Simulation

To convert estimated pairwise Time Differences of Arrival (TDOA) into a two-dimensional source azimuth direction θ, we formulate a hardware-friendly delay-to-angle reconstruction pipeline. This section details the mathematical derivation of our least-squares angle solver, its fixed-point quantization for FPGA deployment, the GCC-PHAT synthetic acoustic simulator, and the evaluation framework.

22.7.1 Least-Squares Direction Reconstruction

For an equilateral array with side length $d = 8$ cm and position vectors $P = [p_0, p_1, p_2]^T \in \mathbb{R}^{3\times 2}$, the differential path length vector $b \in \mathbb{R}^3$ for microphone pairs $(0, 1)$,

EE2000
#4

Table 12. Preliminary Performance Summary of the Revised 16-Bit Correlation Model

Category / Parameter	Value / Rate
SUMMARY STATS	
Total Cases Tested	432
Total Passed	96
Total Failed	336
Overall Accuracy	22.22%
ACCURACY BY RT60 (Echo Level)	
RT60 = 0.00 s	96 / 144 (66.67%)
RT60 = 0.25 s	0 / 144 (0.00%)
RT60 = 0.50 s	0 / 144 (0.00%)
ACCURACY BY SNR (Noise Level)	
SNR = 30 dB	32 / 108 (29.63%)
SNR = 20 dB	34 / 108 (31.48%)
SNR = 10 dB	26 / 108 (24.07%)
SNR = 00 dB	4 / 108 (3.70%)

$(1, 2)$, and $(2, 0)$ relates to the planar acoustic unit vector $u = [\cos\theta, \sin\theta]^T$ through the spatial design matrix $M \in \mathbb{R}^{3\times 2}$:

$$M = \begin{bmatrix} p_0 - p_1 \\ p_1 - p_2 \\ p_2 - p_0 \end{bmatrix}, \qquad b = \frac{c \cdot \boldsymbol{\tau}}{f_s}, \tag{20}$$

where $\boldsymbol{\tau} = [\tau_{01}, \tau_{12}, \tau_{20}]^T$ represents the delay vector in samples, $c = 343$ m/s is the acoustic propagation speed, and $f_s = 48$ kHz is the sampling frequency.

Because three pairwise delays yield an overdetermined linear system $Mu = b$ with two unknowns, we compute the optimal direction vector $\hat{u}$ minimizing the spatial residual error via the pseudoinverse least-squares solution:

$$\hat{u} = (M^T M)^{-1} M^T b = K \cdot \boldsymbol{\tau}, \quad K = (M^T M)^{-1} M^T \frac{c}{f_s}. \tag{21}$$

The azimuth angle in degrees is subsequently extracted using the two-argument arctangent:

$$\hat{\theta} = \operatorname{atan2}(\hat{u}_y, \hat{u}_x) \pmod{360^\circ}. \tag{22}$$

Consistency and Reliability Check. In ideal acoustic conditions, physical wave propagation dictates that closed-loop pairwise delays sum identically to zero ($\tau_{01} + \tau_{12} + \tau_{20} = 0$). To detect multipath corruption, acoustic echo, or severe noise degradation, a reliability flag is asserted if the delay sum remains within a predefined tolerance:

$$\text{Reliable} \iff \left| \sum_{k=1}^{3} \tau_k \right| \leq \text{SUM_TOL} \tag{23}$$

(with SUM_TOL = 1.0 sample)

22.7.2 Fixed-Point FPGA Realization

To evaluate hardware feasibility on an FPGA without floating-point units, the matrix transformation in Eq. (21) is converted to fixed-point integer arithmetic. The pre-calculated geometry transformation matrix $K \in \mathbb{R}^{2\times 3}$ is quantized using $m_{\text{frac}} = 14$ fractional bits, while input sample lags are represented with $\text{lag}_{\text{frac}} = 4$ fractional bits:

$$K_q = \lfloor K \cdot 2^{m_{\text{frac}}} \rceil, \qquad l_q = \lfloor \boldsymbol{\tau} \cdot 2^{\text{lag}_{\text{frac}}} \rceil. \tag{24}$$

The direction vector is computed using integer matrix multiplication:

$$u_q = K_q \cdot l_q \quad \in \mathbb{Z}^{2\times 1}. \tag{25}$$

Because any uniform scaling factor cancels out in $\operatorname{atan2}(u_{q,y}, u_{q,x})$, the resulting azimuth angle $\hat{\theta}$ requires only basic integer multiplication and accumulators, followed by a CORDIC-based or lookup-table atan2 stage (as discussed in Section 13).

22.7.3 Performance Characterization

To evaluate accuracy limitations, we conducted two experimental sweeps at an SNR of 20 dB over 4096-sample windows.

1. **Angular Accuracy Sweep (Float vs. Fixed-Point):** Evaluates the RMS direction error across true azimuth angles $\theta \in [0^\circ, 360^\circ)$ in 5° increments, comparing integer sample lags against sub-sample parabolic interpolation across floating-point and 14-bit/4-bit fixed-point decoding models.

2. **Array Aperture Sweep:** Evaluates overall mean RMS directional error as a function of equilateral microphone spacing $d \in [2\text{ cm}, 20\text{ cm}]$.

22.7.4 Simulation Plots

The above analysis was simulated using `delay_to_angle.py` and plotted as below.

We see that parabolic sub-sample interpolation substantially mitigates discretization errors compared to raw integer lags. Furthermore, it confirms that our fixed-point FPGA quantization (14-bit matrix coefficients, 4-bit fractional lags) achieves near-identical accuracy to the ideal floating-point implementation.

Also we can see that larger array dimensions increase differential time-of-arrival (TDOA) resolution and significantly lower overall directional error, while keeping the considerations of the discussion surrounding Eq. (15) in mind.

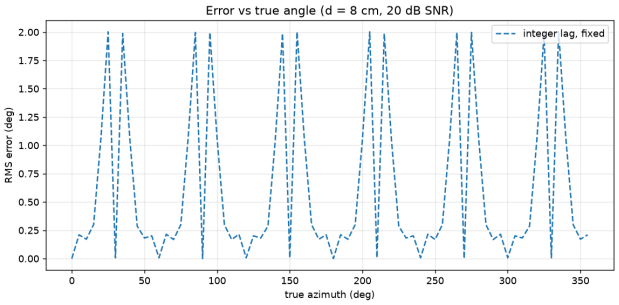


Figure 6. Error Vs Angle Integer-Lag Fixed Point

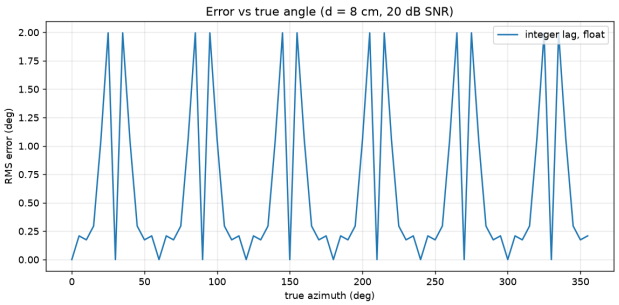


Figure 7. Error Vs Angle Integer-Lag Floating Point

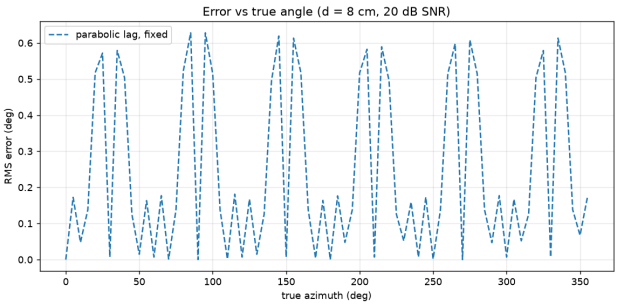


Figure 8. Error Vs Angle Parabolic-Lag Fixed Point

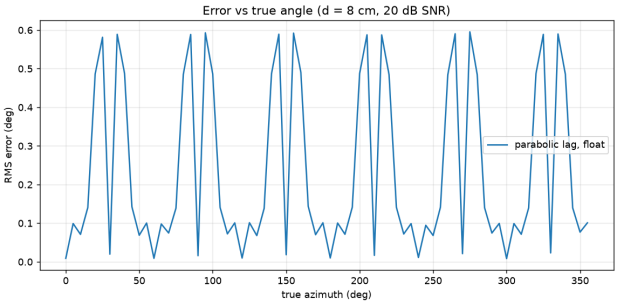


Figure 9. Error Vs Angle Parabolic-Lag Floating Point

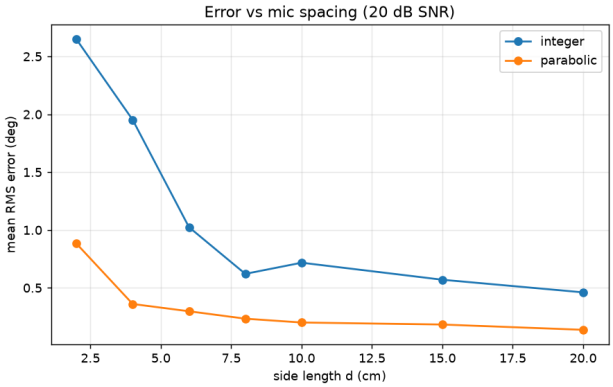


Figure 10. Mean RMS angular error vs. microphone array side length d.

References

[1] Jont B. Allen and David A. Berkley. Image method for efficiently simulating small-room acoustics. *The Journal of the Acoustical Society of America*, 65(4):943–950, Apr. 1979. 10

[2] D. Ayllón, R. Gil-Pita, and M. Rosa-Zurera. Design of microphone arrays for hearing aids optimized to unknown subjects. *Signal Process.*, 93(11):3239–3250, Nov. 2013. 3

[3] Juan Azcarreta Ortiz. Pyramic array: An fpga based platform for many-channel audio acquisition. Master's thesis, École Polytechnique Fédérale de Lausanne, Lausanne, Switzerland, 2016. 2, 4

[4] Tanmay Biswas, Sudhindu Bikash Mandal, Debasri Saha, and Amlan Chakrabarti. Coherence based dual microphone speech enhancement technique using fpga. *Microprocessors and Microsystems*, 55:111–118, 2017. 2, 4

[5] A. Brutti, M. Omologo, and P. Svaizer. Comparison between different sound source localization techniques based on a real data collection. In *Proc. Hands-Free Speech Commun. Microphone Arrays (HSCMA)*, pages 69–72, 2008. 3, 10, 15

[6] Jingdong Chen, J. Benesty, and Yiteng Huang. Robust time delay estimation exploiting redundancy among multiple microphones. *IEEE Transactions on Speech and Audio Processing*, 11(6):549–557, Nov. 2003. 6, 7

[7] Jingdong Chen, J. Benesty, and Yiteng Huang. Robust time delay estimation exploiting spatial correlation. In *2003 IEEE International Conference on Acoustics, Speech, and Signal Processing, 2003. Proceedings. (ICASSP '03).*, volume 5 of *ICASSP-03*, page V–481–4. IEEE, 2003. 7

[8] J. Choi, J. Kim, and S. Im. FB-WKDE: A robust sound source localization approach for TDOA estimation in noisy environments. *IEEE Access*, 13:177745–177759, 2025. 3

[9] I. Céspedes, Y. Huang, J. Ophir, and S. Spratt. Methods for estimation of subsample time delays of digitized echo signals. *Ultrasonic Imaging*, 17(2):142–171, Apr. 1995. 7, 8

[10] Bruno Da Silva, An Braeken, and Abdellah Touhafi. FPGA-based architectures for acoustic beamforming with microphone arrays: Trends, challenges and research opportunities. *Computers*, 7(3):41, 2018. 1, 2, 4

[11] Bruno da Silva, Laurent Segers, An Braeken, Kris Steenhaut, and Abdellah Touhafi. Design exploration and performance strategies towards power-efficient fpga-based architectures

for sound source localization. *Journal of Sensors*, 2019:1–27, 2019. 2, 3

[12] Mojtaba Farmani, Svend Feldt, and Jesper Jensen. Beamforming using virtual microphones for hearing aid applications. In *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 15552–15556, 2026. 3

[13] P. Folkeard, N. S. Jensen, H. Kamkar Parsi, S. Bilert, and S. Scollie. Hearing at the mall: Multibeam processing technology improves hearing group conversations in a real-world food court environment. *Am. J. Audiol.*, 33(3):782–792, Sept. 2024. 3

[14] A. Ganguly, C. Reddy, Y. Hao, and I. Panahi. Improving sound localization for hearing aid devices using smartphone assisted technology. In *Proc. 2016 IEEE Int. Workshop Signal Process. Syst. (SiPS)*, pages 165–170, 2016. 3

[15] E. Grinstein, E. Tengan, B. Çakmak, T. Dietzen, L. Nunes, T. van Waterschoot, M. Brookes, and P. A. Naylor. Steered response power for sound source localization: A tutorial review. *EURASIP J. Audio, Speech, Music Process.*, 2024(1):59, 2024. 4

[16] François Grondin et al. Fast cross-correlation for tdoa estimation on small aperture microphone arrays. In *arXiv preprint*, 2022. 1

[17] H.-Y. Gu and S.-S. Yang. A sound-source localization system using three-microphone array and crosspower spectrum phase. In *Proc. 2012 Int. Conf. Machine Learning and Cybernetics*, pages 1732–1736, Xi'an, China, 2012. 3, 9, 12

[18] X. Hu, J. Zhang, H. Ma, J. Shi, and Y. Du. Robust real-time DOA estimation for outdoor vehicle acoustic sources using dynamic-pruning GCC-PHAT and adaptive forgetting factor OPAST-MUSIC. *Sensors*, 26(13):4281, 2026. 4

[19] T. Inoue, Y. Ikeda, and Y. Oikawa. Hat-type hearing aid system with flexible sound directivity pattern. *Acoust. Sci. Technol.*, 39(1):22–29, Jan. 2018. 3

[20] G. Jacovitti and G. Scarano. Discrete time techniques for time delay estimation. *IEEE Transactions on Signal Processing*, 41(2):525–533, 1993. 8

[21] Norish Camelia Jamaludin, Ili Shairah Abdul Halim, Siti Lailatul Mohd Hassan, and Wan Fazlida Hanim Abdullah. Real-time FPGA-based sound source localization. In *2024 IEEE 14th Symposium on Computer Applications & Industrial Electronics (ISCAIE)*. IEEE, 2024. 1, 2

[22] Seunghun Jin; Dongkyun Kim; Hyung Soon Kim; Chang Hoon Lee; Jong Suk Choi; Jae Wook Jeon. Real-time sound source localization system based on FPGA. In *The IEEE International Conference on Industrial Informatics (INDIN 2008)*., 2008. 1, 2

[23] Y. Kadhim and A. Khalifeh. Performance evaluation and comparative analysis for widely used sound source localization algorithms. In *Proc. 2022 5th Int. Conf. Communications, Signal Processing, and their Applications (ICCSPA)*, 2022. 3

[24] U. Klein and T. Q. Võ. Direction-of-arrival estimation using a microphone array with the multichannel cross-correlation method. In *Proc. 2012 Int. Symp. Communications and Information Technologies*, pages 251–256, 2012. 4, 6, 14, 16

[25] C. Knapp and G. Carter. The generalized correlation method for estimation of time delay. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 24(4):320–327, 1976. 1, 6, 8, 10

[26] A. Kovalyov, K. Patel, and I. Panahi. Accurate real-time estimation of 2-dimensional direction of arrival using a 3-microphone array. In *Proc. 2023 57th Asilomar Conf. Signals, Systems, and Computers*, pages 1179–1183, 2023. 1, 2, 4, 9, 10

[27] Konrad Kowalczyk, Szymon Wozniak, Tomasz Chyrowicz, and Roman Rumian. Embedded system for acquisition and enhancement of audio signals. In *2016 Signal Processing: Algorithms, Architectures, Arrangements, and Applications (SPA)*, pages 68–71, Poznan, Poland, 2016. 2, 4

[28] Mauricio Kugler Teemu Tossavainen Susumu Kuroyanagi. Design of a compact sound localization device on a standalone FPGA-based platform. In *IEICE TRANS. INF. & SYST*, 2016. 1, 2

[29] Krzysztof Kakol and Bożena Kostek. A study on signal processing methods applied to hearing aids. In *2016 Signal Processing: Algorithms, Architectures, Arrangements, and Applications (SPA)*, pages 219–224, 2016. 1

[30] T.I. Laakso, V. Valimaki, M. Karjalainen, and U.K. Laine. Splitting the unit delay [fir/all pass filters design]. *IEEE Signal Processing Magazine*, 13(1):30–60, 1996. 8

[31] Ákos Lédeczi, András Nádas, Péter Völgyesi, György Balogh, Branislav Kusy, János Sallai, Gábor Pap, Sebestyén Dóra, Károly Molnár, Miklós Maróti, and Gyula Simon. Countersniper system for urban warfare. *ACM Transactions on Sensor Networks*, 1(2):153–177, 2005. 2, 4

[32] Chun-Lin Liu and P. P. Vaidyanathan. One-bit sparse array doa estimation. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, page 3126–3130. IEEE, Mar. 2017. 7

[33] J. A. Lopez-Olvera, H. M. Perez-Meana, E. Garcia-Rios, E. Escamilla-Hernandez, J. M. Carrichi-Chavez, and J. Betancourt-Martinez. Multichannel acoustic beamforming for speaker localization and DOA-based tracking. *Electronics*, 15(15):3410, 2026. 4

[34] D.L. Maskell and G.S. Woods. The discrete-time quadrature subsample estimation of delay. *IEEE Transactions on Instrumentation and Measurement*, 51(1):133–137, 2002. 8

[35] Duy Nguyen, Parham Aarabi, and Ali Sheikholeslami. Real-time sound localization using field-programmable gate arrays. In *Proceedings of the 2003 IEEE International Conference on Multimedia and Expo (ICME)*, volume 2, pages 6–10, 2003. 1, 2

[36] U. T. K. Nguyen and T. V. Pham. Performance assessment of generalized cross-correlation based algorithms for multisource point-based localization and detection. In *Proc. 2011 Int. Conf. Advanced Technologies for Communications (ATC)*, pages 303–306, 2011. 3, 10, 11

[37] Lucian Petrica. An evaluation of low-power microphone array sound source localization for deforestation detection. *Applied Acoustics*, 113:162–169, 2016. 2, 4

[38] Lucian Petrica and Gheorghe Stefan. Energy-efficient wsn architecture for illegal deforestation detection. *International*

Journal of Sensors and Sensor Networks, 3(3):24–30, 2015. 2, 4

[39] T. A. Powers and V. Hamacher. Three-microphone instrument is designed to extend benefits of directionality. *Hearing J.*, 55(10):38–45, Oct. 2002. 3, 14

[40] H. Puder. Adaptive signal processing for interference cancellation in hearing aids. *Signal Process.*, 86(6):1239–1253, June 2006. 3

[41] K. Samtani, J. Thomas, S. P. Deepu, and S. David S. Area and power optimised ASIC implementation of adaptive beamformer for hearing aids. In *Proc. 2017 IEEE Biomed. Circuits Syst. Conf. (BioCAS)*, pages 1–4, Turin, Italy, Oct. 2017. 3, 4

[42] K. Samtani, J. Thomas, G. A. Varma, S. David S., and S. P. Deepu. FPGA implementation of adaptive beamforming in hearing aids. In *Proc. 2017 39th Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. (EMBC)*, pages 2239–2242, Seogwipo, South Korea, July 2017. 1, 2, 3, 4

[43] R. Scheibler, E. Bezzam, and I. Dokmanić. Pyroomacoustics: A Python package for audio room simulation and array processing algorithms. In *Proc. IEEE ICASSP*, pages 351–355, 2018. 10

[44] Tomyslav Sledevič and Raimond Laptik. An evaluation of hardware-software design for sound source localization based on soc. In *Proceedings of the 2017 Open Conference of Electrical, Electronic and Information Sciences (eStream)*, pages 1–4, Vilnius, Lithuania, 2017. 2, 4

[45] Tomoya Takagi, H. Noguchi, Koji Kugata, Masahiko Yoshimoto, and H. Kawaguchi. Microphone array network for ubiquitous sound acquisition. In *2010 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1474–1477, Dallas, TX, USA, 2010. 2, 4

[46] S. Tokgöz, A. Kovalyov, and I. Panahi. Real-time estimation of direction of arrival of speech source using three microphones. In *Proc. 2020 IEEE Workshop Signal Process. Syst. (SiPS)*, pages 1–5, 2020. 4

[47] S. Tokgoz and I. M. S. Panahi. Robust three-microphone speech source localization using randomized singular value decomposition. *IEEE Access*, 9:157800–157811, 2021. 4

[48] B. D. Van Veen and K. M. Buckley. Beamforming: A versatile approach to spatial filtering. *IEEE ASSP Magazine*, 5(2):4–24, Apr. 1988. 13, 14

[49] J.H. Van Vleck and D. Middleton. The spectrum of clipped noise. *Proceedings of the IEEE*, 54(1):2–19, 1966. 7, 12

[50] Trình Qu´ˆøc Võ and Udo Klein. A soft-processor-based FPGA implementation of a microphone array direction-of-arrival estimator. In *Vietnam National University HCMC.-unpublished*, 2013. 1, 2, 6

[51] D. Ying, J. Li, Y. Feng, and Y. Yan. Direction of arrival estimation based on weighted minimum mean square error. In *Proc. 2013 IEEE China Summit and Int. Conf. Signal and Information Processing (ChinaSIP)*, pages 318–321, 2013. 3, 6